Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt zespołowy – laboratorium 2

Implementacja bazy danych szeregów czasowych w języku Rust

Architektura systemu i szczegółowy projekt techniczny

Autorzy: Hubert Błonowski
Jan Zaborski

Promotor: mgr inż. Paweł Sobótka

Wersja dokumentu: 1.2
Data: 24 października 2026 r.

`https://github.com/janzaborski/rust-tsdb`

Streszczenie

Dokument przedstawia projekt jednowęzłowej bazy danych szeregów czasowych napisanej w języku Rust. Określa podział systemu na moduły, model danych, komunikację przez HTTP, podzbiór języka PromQL oraz integrację z Grafaną. Szczegółowo opisuje buforowanie pomiarów, dziennik wyprzedzający zapis (WAL), format niezmiennych segmentów, indeksowanie i odzyskiwanie po awarii procesu. Jedna struktura TSMap udostępnia szeregi TS z osobnymi blokadami. Rotacja bloków pozwala kontynuować zapis podczas zrzutu na dysk. Buforowany WAL jest obsługiwany przez flusher i syncer; każde żądanie wybiera poziom trwałości None, Written albo Synced. Definiuje kontrakty, na podstawie których można niezależnie implementować silnik przechowywania, obsługę zapytań i warstwę HTTP.

Jest to **specyfikacja architektury docelowej**, a nie opis stanu repozytorium. Zaprojektowane mechanizmy i podane wartości wydajnościowe wymagają weryfikacji podczas implementacji. Dokument z laboratorium 1 stanowi punkt wyjścia; doprecyzowania jego wymagań wskazano jawnie.

Historia zmian

Wersja	Data	Autorzy	Zakres zmiany
1.0	24.10.2026	Hubert Błonowski, Jan Zaborski	Projekt architektury, kontrakty modułów i API, formaty plików, diagramy, wizja interfejsu oraz plan weryfikacji.
1.1	24.10.2026	Hubert Błonowski, Jan Zaborski	TSMap i TS inspirowane Gorilla, współbieżne zapisy bez shardów, rotacja bloków i WAL, flush w tle oraz uaktualnione recovery i testy.
1.2	24.10.2026	Hubert Błonowski, Jan Zaborski	Buforowany WAL, flusher i syncer, bajtowy LSN, poziomy trwałości, niezależna rotacja plików oraz dostosowane API, checkpoint i recovery.

Spis treści

Streszczenie **i**

Historia zmian **i**

1 Cel, zakres i decyzje architektoniczne **1**

1.1 Cel projektu i odbiorcy dokumentu . . . 1
1.2 Granice systemu . . . 1
1.3 Doprecyzowania względem laboratorium 1 . . . 1
1.4 Rejestr głównych decyzji . . . 2

2 Architektura systemu **2**

2.1 Kontekst i topologia wdrożenia . . . 2
2.2 Warstwy, moduły i zależności . . . 3
2.3 Współbieżność i spójność . . . 4

3 Model danych i kontrakty modułów **5**

3.1 Tożsamość szeregu i reprezentacja punktu . . . 5
3.2 Struktury TSMap, TS i bloki . . . 5
3.3 Duplikaty i atomowość paczek . . . 6

3.4 Kontrakty wewnętrzne . . . 7
3.5 Podział odpowiedzialności zespołu . . . 8

4 Zapis, buforowanie i obsługa opóźnionych danych 8
4.1 Algorytm przyjęcia paczki . . . 8
4.2 Warunki przyjmowania out-of-order . . . 8
4.3 Organizacja i limit pamięci . . . 9
4.4 Wyzwalanie i przebieg zrzutu . . . 10

5 Organizacja danych na dysku 10
5.1 Katalog i wspólne reguły formatu . . . 10
5.2 Bufor WAL, flusher i syncer . . . 11
5.3 Format dziennika i bajtowy LSN . . . 13
5.4 Segment danych . . . 14
5.5 Manifest i katalog szeregów . . . 15
5.6 Indeksy w pamięci i plan odczytu . . . 16

6 Checkpoint i odzyskiwanie po awarii 16
6.1 Trwałe opublikowanie generacji . . . 16
6.2 Algorytm uruchomienia . . . 17
6.3 Scenariusze awarii i granice gwarancji . . . 18
6.4 Cykl życia procesu . . . 19

7 Język i wykonywanie zapytań 19
7.1 Obsługiwana składnia . . . 19
7.2 Czas obliczenia i wybór próbek . . . 20
7.3 Przykład rozstrzygający granice . . . 21
7.4 Plan wykonania i limity pracy . . . 21

8 Komunikacja i publiczne API 22
8.1 Transport i konwencje . . . 22
8.2 Zapis pojedynczy i paczkowy . . . 22
8.3 Zapytania chwilowe i zakresowe . . . 23
8.4 Metadane i stan serwera . . . 24
8.5 Błędy i przeciążenie . . . 24

9 Technologie, konfiguracja i środowisko 25
9.1 Wybór technologii . . . 25
9.2 Konfiguracja procesu . . . 27
9.3 Sieć, trwałość i granica dostępu . . . 28
9.4 Diagnostyka . . . 28

10 Wizja interfejsu użytkownika i demonstracja 29
10.1 Dashboard Grafany . . . 29
10.2 Profil standardowego źródła Prometheus . . . 30
10.3 Scenariusz demonstracji . . . 30

11 Weryfikacja architektury i kryteria odbioru 30
11.1 Powiązanie z wymaganiami . . . 30
11.2 Scenariusze poprawności . . . 31
11.3 Profil pomiarów wydajności i zasobów . . . 33

11.4 Ryzyka i warunki akceptacji projektu 34

A Słownik **35**

B Budowanie dokumentu **35**

Źródła **36**

1 Cel, zakres i decyzje architektoniczne

1.1 Cel projektu i odbiorcy dokumentu

System ma przyjmować pomiary przesyłane przez aplikacje, trwale przechowywać je we własnym silniku oraz udostępniać historyczne dane i agregacje czasowe. Użytkownik analizuje wyniki w Grafanie. Głównym problemem inżynierskim jest pogodzenie trwałości potwierdzonych zapisów, ograniczonej pamięci i odczytu danych rozdzielonych między RAM a dysk.

Dokument jest przeznaczony dla autorów implementacji, promotora i osoby weryfikującej projekt. Normatywne sformułowania opisują zachowanie, które implementacja ma osiągnąć. Przykłady przedstawiają projektowane kontrakty; nie stanowią wyników uruchomienia obecnego prototypu.

1.2 Granice systemu

Zakres obejmuje własny silnik, serwer HTTP, dokładne dopasowanie etykiet, pięć agregacji w czasie i demonstrację w Docker Compose. Proces bazy przechowuje dane na lokalnym systemie plików. Grafana oraz generatory są zewnętrznymi klientami, a nie częściami silnika.

W tej wersji nie projektuje się replikacji, klastra, kompresji, automatycznej retencji, kompaktowania segmentów, alertowania, skanowania źródeł `/metrics` ani osobnego frontendu. Katalog danych rośnie do chwili usunięcia całego środowiska przez operatora. Operacyjna kopia zapasowa wymaga zatrzymania procesu i skopiowania całego katalogu.

1.3 Doprecyzowania względem laboratorium 1

Wymaganie	Przyjęta interpretacja	Konsekwencja projektowa
FR-01	Paczka jest jednostką atomowego zapisu.	Niepoprawny punkt odrzuca całą paczkę. Błąd transportu po utrwaleniu nie oznacza wycofania operacji.
FR-03	Widoczność w head i wybrany poziom trwałości.	Domyślne Written zapewnia zapis całej ramki do systemu operacyjnego przed odpowiedzią. Synced wymaga synchronizacji nośnika. Jawnie wybrane None dopuszcza utratę potwierdzonych danych po awarii procesu.
FR-04	Zapytanie zakresowe wielokrotnie oblicza wyrażenie.	Sam selektor wybiera ostatnią próbkę; nie wykonuje automatycznej agregacji. Agregację wskazuje funkcja w wyrażeniu.
FR-08	Czas buforowania jest liczony od przyjęcia najstarszej próbki w bieżącej generacji.	Zrzut obejmuje całą generację; młodsze próbki mogą opuścić RAM wcześniej. Limit pamięci może wymusić wcześniejszy zrzut.
FR-09	Out-of-order dotyczy wyłącznie otwartego bufora.	Przyjęcie wymaga zarówno zachowania okna opóźnienia, jak i przekroczenia granicy zamknięcia szeregu. Samo zmieszczenie się w oknie nie wystarcza.
NFR-03	Jednostki pamięci są bajtowe.	Cel wynosi RSS $\leq$ 1 GiB, a nie 1 Gb. Około 288 MB to surowe próbki, bez pojemności zapasowej kontenerów, indeksów i obsługi zapytań.

Wymaganie	Przyjęta interpretacja	Konsekwencja projektowa
NFR-04	Gwarancja obejmuje awarię procesu przy poprawnie działającym systemie operacyjnym i nośniku.	Kryterium braku utraty dotyczy Written i Synced; None jest mierzone oddzielnie. Błąd I/O w obsłudze WAL kończy proces, a nie daje sukcesu oczekującym klientom. Uszkodzenie nośnika jest poza gwarancją.

1.4 Rejestr głównych decyzji

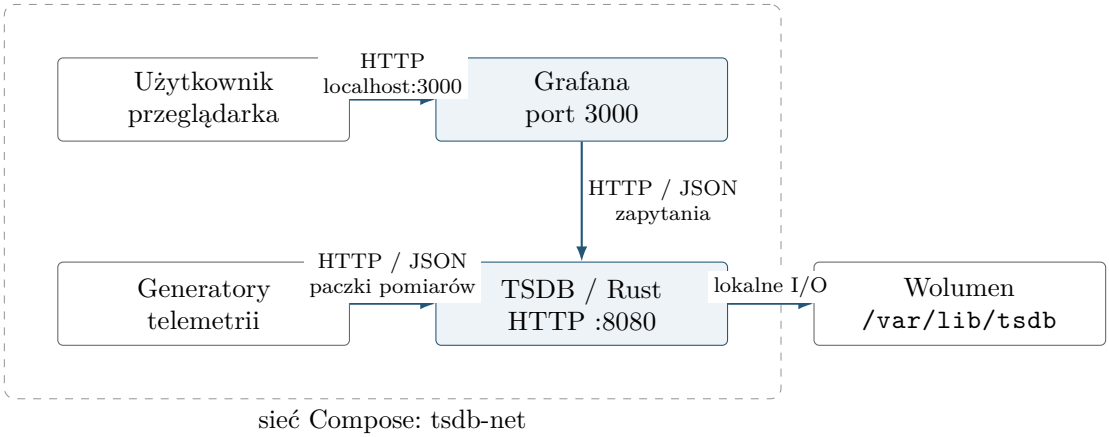
ID	Decyzja	Uzasadnienie i koszt
DA-01	Modularny monolit w Rust.	Brak komunikacji sieciowej wewnątrz silnika; prostsze transakcje i diagnostyka. Skalowanie ograniczone do jednego węzła.
DA-02	Jedna TSMap i synchronizacja każdego TS.	Wiele operacji zapisu może równolegle zmieniać rozłączne szeregi. Katalog ma krótkie sekcje krytyczne; projekt nie wymaga shardów.
DA-03	Aktywne i zamrożone bloki bez kompresji.	Uporządkowane wektory obsługują out-of-order, a niezmienne bloki pozostają dostępne podczas zapisu na dysk.
DA-04	Wspólne generacje, flush w tle.	Krótka bariera zamraża head i ustala granicę checkpointu. Zapis segmentu nie zatrzymuje nowych zapisów; trwa najwyżej jeden flush.
DA-05	Grafana jako jedyny interfejs graficzny.	Wykorzystanie standardowego źródła Prometheus; konieczne testy zgodności z ustaloną wersją Grafany.
DA-06	Brak modyfikacji zamkniętych zakresów.	Proste indeksy i niepokrywające się przedziały segmentów danego szeregu. Starszy punkt może zostać odrzucony po wcześniejszym zrzucie.
DA-07	Jeden logiczny WAL z rotowanymi plikami.	Żądania dopisują do bufora RAM. Flusher zapisuje pliki, syncer synchronizuje je. Oczekiwanie na trwałość jest poza blokadami TS.
DA-08	Atomowe paczki i spójny widok wybranych szeregów.	Blokady TS są pobierane w kolejności identyfikatorów. Duża paczka lub zapytanie może chwilowo ograniczyć równoległość.
DA-09	Written jako domyślny poziom trwałości.	None i Synced są wybierane per żądanie. Widoczność danych jest oddzielona od zapisu do systemu operacyjnego i synchronizacji.

2 Architektura systemu

2.1 Kontekst i topologia wdrożenia

System działa na jednym komputerze. Przeglądarka komunikuje się z Grafaną; to serwer Grafany wysyła zapytania do bazy. Generatory dostarczają pomiary przez własne API zapisu HTTP JSON. W środowisku nie działa serwer Prometheus.

Proces bazy ma wyłączny dostęp do katalogu danych. Blokada systemowa pliku `LOCK`, utrzymywana przez otwarty deskryptor, uniemożliwia uruchomienie drugiego procesu na tym samym katalogu.

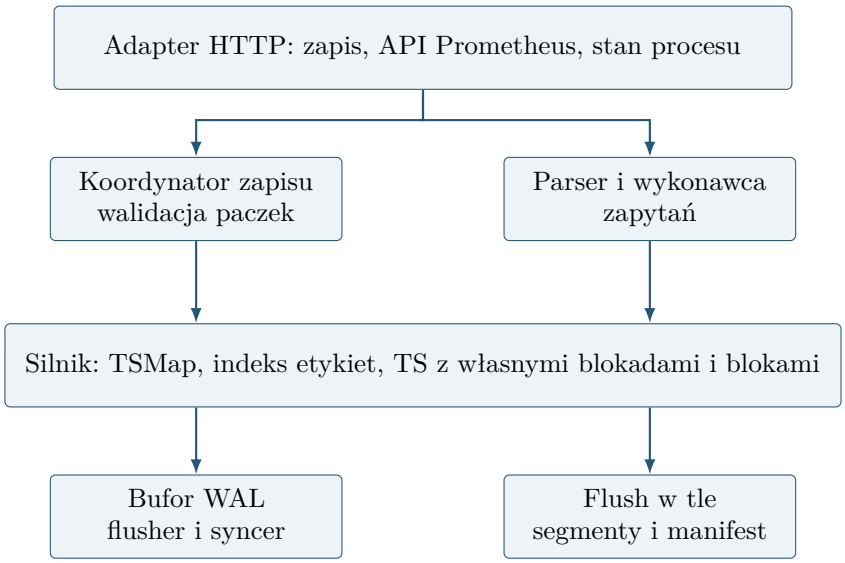


Rysunek 1: Kontekst i kontenery środowiska demonstracyjnego. Wolumen jest zasobem hosta.

Istnienie samego pliku po awarii nie oznacza aktywnej blokady. Systemy plików sieciowych nie są platformą docelową.

2.2 Warstwy, moduły i zależności

Warstwa HTTP odpowiada za protokół, warstwa aplikacyjna za operacje zapisu i zapytań, a silnik za dane i pliki. Parser zależy wyłącznie od typów języka. Silnik nie zna kodów HTTP, Grafany ani składni PromQL.



Rysunek 2: Zależności logiczne. Konfiguracja i diagnostyka są współdzielonymi usługami pomocniczymi.

Moduł	Odpowiedzialność	Interfejs i zależności
HTTP	Dekodowanie, limity żądań, mapowanie błędów i kodowanie odpowiedzi.	Wywołuje zapis lub wykonawcę zapytań; używa Axum i Serde.
Model i walidacja	Kanonizacja klucza szeregu, sprawdzanie typów, limity etykiet i wartości.	Typy wspólne, bez zależności od HTTP i plików.

Moduł	Odpowiedzialność	Interfejs i zależności
Koordynator zapisu	Pula operacji, uporządkowane blokady TS, walidacja paczek i rezerwacja pamięci.	TSMap, TS, usługa WAL i bariera generacji.
TSMap i indeks	Klucz ↔ uchwyt TS, rejestracja szeregów i indeks etykiet.	Metadane pod krótką blokadą katalogu.
TS i bloki	Posortowane próbki, duplikaty, granice czasu, blok aktywny i bloki zamrożone.	Osobna blokada każdego TS; pamięć rozliczana globalnie.
WAL	Bufor RAM, bajtowy LSN, granice written/synced i rotacja plików.	Mutex bufora, flusher, syncer i kanały watch; bez blokad TS w zadaniach I/O.
Magazyn segmentów	Zapis bloków, indeksy czasowe, manifest i checkpoint.	Lokalne pliki, sumy CRC, koordynator zrzutu.
Parser	Tekst → AST lub błąd z pozycją.	Bez dostępu do danych; typy selektora, funkcji i czasu.
Wykonawca zapytań	Dobór szeregów, plan odczytu, agregacje, limity pracy.	AST i interfejs odczytu; bez bezpośredniego dostępu do WAL.
Uruchomienie i diagnostyka	Konfiguracja, blokada katalogu, recovery, gotowość, logi i zakończenie.	Zarządza cyklem życia pozostałych modułów.

2.3 Współbieżność i spójność

Tokio obsługuje sieć, a ograniczona pula robocza wykonuje część pamięciową zapisu, domyślnie cztery zadania równolegle. Każdy TS ma własny `RwLock`. Rozłączne szeregi mogą być walidowane i aktualizowane równolegle. Jeden logiczny WAL porządkuje ramki podczas krótkiego dopisywania do bufora `Mutex<Vec<u8>>`.

Kolejność blokad zapisu: jeżeli potrzebne są nowe szeregi, najpierw mutex rejestracji, następnie współdzielona bariera generacji head, a potem wyłączne blokady TS paczki w rosnącej kolejności `SeriesId`. Odczytanie uchwytów z TSMap kończy się przed pobieraniem blokad TS. Pod tymi blokadami przeprowadza się walidację, rezerwuje alokacje, koduje ramkę, dopisuje ją do bufora WAL i aktualizuje head. Przed zwolnieniem którejkolwiek blokady publikuje się całą paczkę.

Mutex bufora WAL chroni kopię kompletnej ramki i przydział jej LSN. Nie obejmuje aktualizacji head, operacji plikowych ani oczekiwania na trwałość. Dopisanie może czekać na mutex lub budżet pamięci, ale nie wykonuje I/O. Nie używa się terminu lock-free.

Po publikacji zwalnia się blokady TS, mutex rejestracji, barierę oraz miejsce w puli roboczej. Dopiero wtedy obsługa HTTP asynchronicznie czeka na wybrany poziom Written lub Synced. None odpowiada bez tego oczekiwania. Zawieszone oczekiwanie nie zajmuje wątku roboczego ani blokad potrzebnych kolejnym zapisom i zamrażaniu head.

Flusher zapisuje przejęte bufory do plików, a syncer wykonuje `fdatasync`. Nie pobierają blokad TS ani bariery head. Udostępniają przez kanały watch monotoniczne granice zapisu i synchronizacji. Szczegóły wyznacza sekcja 5.2.

Odczyt spójnego widoku: zapytanie wybiera z TSMap uchwyty i wersję katalogu, zwalnia blokadę mapy, następnie pobiera współdzielone blokady wszystkich wybranych TS w tej samej rosnącej kolejności. Sprawdza atomowy licznik wersji katalogu; jeśli w międzyczasie zarejestrowano szereg, zwalnia blokady i ponawia wybór w ramach timeoutu. Nie pobiera ponownie blokady TSMap, trzymając blokady TS. Kopiuje potrzebne aktywne próbki i uchwyty do niezmiennych bloków oraz segmentów, po czym zwalnia blokady. Oczekiwanie na blokady używa `try_read` z kontrolą timeoutu.

Widok odpowiada jednej chwili stanu wybranych szeregów i nie zawiera części paczki. Całe zapytanie zakresowe korzysta z tego samego widoku. Może zobaczyć paczkę, której klient nadal oczekuje na Written lub Synced: **widoczność w head i trwałość są odrębnymi właściwościami**. Nie opisuje się widoku pojedynczym globalnym LSN, ponieważ rozłączne publikacje mogą kończyć się poza kolejnością WAL. Odczyt po odpowiedzi sukcesu widzi zapis w odpowiednim zakresie.

Zamrażanie head uzyskuje wyłączną barierę i czeka tylko na zakończenie części pamięciowych rozpoczętych zapisów. Ustala graniczny LSN, zamraża aktywne bloki i zwalnia barierę bez czekania na dysk. Rotacja plików WAL jest odrębną czynnością flushera. Zapis segmentu danych i manifestu wykonuje zadanie flush w tle; nowe żądania w tym czasie aktualizują nowe aktywne bloki.

3 Model danych i kontrakty modułów

3.1 Tożsamość szeregu i reprezentacja punktu

Klucz szeregu składa się z nazwy metryki oraz uporządkowanej listy par etykiet. Klucze etykiet są unikalne; sortowanie jest leksykograficzne według bajtów UTF-8. Nie normalizuje się wielkości liter ani Unicode. Porównanie pełnego klucza rozstrzyga tożsamość również przy kolizji funkcji skrótu.

Typ / pole	Reprezentacja	Reguła
`SeriesKey`	Metryka i posortowane pary napisów.	Nazwa metryki: `[a-zA-Z_:][a-zA-Z0-9_:]*`, do 128 bajtów.
Etykieta	Nazwa i wartość UTF-8.	Nazwa: `[a-zA-Z_][a-zA-Z0-9_]*`, do 64 bajtów; wartość do 256 bajtów, maksymalnie 16 etykiet.
Nazwy zastrzeżone	Prefiks `__`.	Nie są dopuszczane w etykietach zapisu. `__name__` jest wirtualną etykietą nazwy metryki w odpowiedziach.
`SeriesId`	`u64`, od 1.	Przydzielany rosnąco przy rejestracji w TSMap; nie jest hashem ani częścią publicznego API.
`Timestamp`	`i64`, milisekundy od epoki Unix.	Zakres przyjmowania: 0 do $2^{53} - 1$ ms, aby zachować dokładność całkowitą w klientach JSON.
`Sample`	`timestamp:i64`, `value:f64`.	16 bajtów w bloku; wartość wejściowa musi być skończona. Normalizacja -0 do $+0$.
`Lsn`	`u64`, od 1.	Pozycja końca rekordu w logicznym strumieniu bajtów WAL, bez nagłówków plików. Rośnie o długość ramki; 0 oznacza pusty strumień.

Pusta wartość etykiety jest kanonizowana przez usunięcie tej etykiety. Dzięki temu porównanie z pustym napisem może objąć również szeregi bez danej etykiety. Powtórzona nazwa klucza w obiekcie JSON jest błędem przed kanonizacją, także gdy parser JSON domyślnie potrafiłby ją nadpisać. Nieznane pola wejściowe są odrzucane. Wartości metryk i etykiet w przykładach stosują konwencje Prometheusa; projekt celowo ogranicza dozwolone nazwy do wskazanej składni ASCII.

3.2 Struktury TSMap, TS i bloki

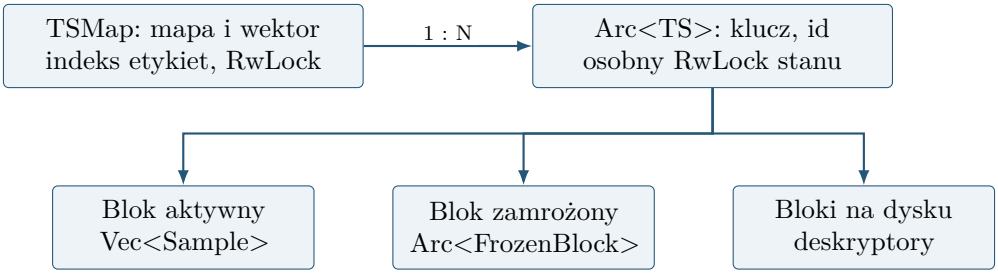
Inspiracją jest oddzielenie katalogu TSmap, synchronizacji szeregu i otwartych oraz zamkniętych bloków z sekcji 4.2 artykułu Gorilla [2]. W projekcie pozostaje jedna TSMap dla całej bazy. Bloki

przechowują niekompresowane próbki; ich rolą jest umożliwienie odczytu i dalszego zapisu podczas flush. Buforowanie dziennika nawiązuje do sekcji 4.3 artykułu, ale projekt jawnie rozróżnia None, Written i Synced. Domyślny tryb nie dopuszcza utraty potwierdzonych danych przy awarii procesu.

```
TSMap {
  by_key: HashMap<SeriesKey, Arc<TS>>,
  by_id: Vec<Arc<TS>>,
  label_index: LabelIndex,
  catalog_version: AtomicU64
}
TS {
  id: SeriesId,
  key: SeriesKey,
  state: RwLock<TSState>
}
TSState {
  latest_time: Option<Timestamp>,
  sealed_through: Option<Timestamp>,
  last_lsn: Lsn,
  active: MutableBlock,
  frozen: Option<Arc<FrozenBlock>>,
  persisted: Vec<BlockDescriptor>
}
```

Mapę, wektor i indeks etykiet chroni jeden `RwLock` katalogu. `by_id` służy do skanowania podczas rotacji, a mapa do wyszukiwania po kluczu. Nie usuwa się szeregów ani nie ponownie wykorzystuje ich identyfikatorów; wektor jest uporządkowaną listą uchwytów, a nie tablicą indeksowaną bezpośrednio wartością ID. Wyszukiwanie po ID jest binarne. Klucz i identyfikator TS są niezmienne. Licznik wersji zwiększa się przy publikacji nowej zawartości katalogu, przed zwolnieniem blokad TS.

Jeżeli paczka wymaga nowych szeregów, mutex rejestracji zapobiega równoległemu przydzieleniu dwóch TS temu samemu kluczowi. Po jego uzyskaniu ponawia się wyszukiwanie; nowe TS są przygotowywane poza publiczną mapą. Po włączeniu pełnej ramki do bufora WAL krótka wyłączna blokada TSMap publikuje je wraz z indeksami, podczas gdy wszystkie TS paczki pozostają zablokowane. Definicje nowych szeregów są w tej samej ramce co ich pierwsze próbki. Dopiero po publikacji paczki inne zadanie może użyć nowego TS, więc jego rekord zawsze będzie późniejszy od definicji. Blokada indeksu nie obejmuje dopisywania do bufora ani oczekiwania na I/O. Paczki dotyczące istniejących szeregów nie pobierają mutexu rejestracji. Nieudana rezerwacja ID może zostawić lukę, ale nie pusty publiczny szereg.



Rysunek 3: Jedna TSMap i niezależne TS. Każdy szereg ma jeden aktywny blok, co najwyżej jeden zamrożony oraz dowolną liczbę deskryptorów dyskowych.

3.3 Duplikaty i atomowość paczek

W jednym szeregu może istnieć najwyżej jedna wartość dla danego znacznika czasu. W otwartym buforze ponowne przesłanie identycznej próbki jest operacją bez zmian; odpowiedź zawiera licznik duplikatów. Inna wartość dla tego samego czasu powoduje konflikt i odrzucenie całej paczki. To

samo dotyczy powtórzeń wewnątrz paczki.

Po zamknięciu zakresu punkt z czasem mniejszym lub równym granicy zamknięcia jest odrzucany bez odczytywania dysku, także jeśli jest identycznym duplikatem. API nie zapewnia więc trwałej idempotencji ponowień przez dowolnie długi czas. Utrata odpowiedzi może pozostawić klienta bez wiedzy o wyniku; wynik można sprawdzić zapytaniem. Nie wprowadza się osobnego rejestru kluczy idempotencji.

Walidacja wyznacza plan zmian dla całej paczki, z uwzględnieniem jej powtórzeń. Trwałość definicji nowych szeregów jest taka sama jak całej ramki. Paczka zawierająca wyłącznie identyczne duplikaty nie tworzy nowej ramki; jako cel oczekiwania bierze największe `last_lsn` szeregów tej paczki. Również wtedy wykonuje oczekiwanie Written lub Synced: ponowienie zapisu None nie może uzyskać mocniejszego potwierdzenia bez osiągnięcia odpowiedniej granicy WAL. Nie jest to obietnica, że wszystkie mniejsze LSN są już widoczne w innych szeregach. Paczki o wspólnym TS zachowują kolejność WAL, a rozłączne paczki nie wymagają wspólnej kolejności publikacji.

3.4 Kontrakty wewnętrzne

Notacja określa granice modułów, bez narzucania osobnych crate'ów.

```
enum Durability { None, Written, Synced }

canonicalize(input: Measurement) -> Result<CanonicalPoint, ValidationError>
parse(text: &str) -> Result<Expr, ParseError>

submit(batch: Vec<CanonicalPoint>, mode: Durability)
  -> Result<WriteReceipt, WriteError>
// WriteReceipt { inserted, duplicates, lsn, durability }
// Sukces: cala paczka jest widoczna i osiagnieto wybrany poziom.

snapshot(selector: &Selector, bounds: TimeBounds,
         budget: &mut QueryBudget) -> Result<ReadView, ReadError>
read_series(view: &ReadView, id: SeriesId,
            bounds: TimeBounds) -> Result<SampleCursor, ReadError>
evaluate(expr: &Expr, grid: EvalGrid, view: &ReadView,
         budget: &mut QueryBudget) -> Result<QueryResult, QueryError>

wal.enqueue(record: EncodedRecord) -> Result<Lsn, AdmissionError>
// Brak I/O; zwraca pozycje konca kompletnej ramki.
wal.wait_for(lsn: Lsn, mode: Durability) -> Result<(), WaitError>
// Wywolywane po publikacji head i zwolnieniu blokad.

freeze_head() -> Result<FrozenGeneration, StorageError>
persist(generation: FrozenGeneration) -> Result<Checkpoint, StorageError>
recover(path: &Path) -> Result<RecoveredState, RecoveryError>
```

`Expr` zawiera selektor, funkcję czasową z oknem albo proste wyrażenie skalarne. `ReadView` przechowuje wersję katalogu, kopię potrzebnych aktywnych próbek i uchwyty do bloków i segmentów. `SampleCursor` zwraca próbki chronologicznie, bez ujawniania źródła. `QueryBudget` śledzi czas, pamięć, próbki i rozmiar wyniku. `FrozenGeneration` zawiera numer generacji head, graniczny LSN, katalog z chwili zamrożenia i uchwyty do zamrożonych bloków.

Błędy walidacji, konfliktu i braku budżetu są zwracane przed enqueue. Po nim nie ma zwykłej ścieżki wycofania: publikacja korzysta z wcześniej zarezerwowanych zasobów. Timeout oczekiwania na trwałość lub utrata połączenia nie wycofują danych. Błąd I/O w WAL kończy proces zgodnie z sekcją 5.2; nie jest mapowany na sukces. Pozostałe kategorie i mapowanie HTTP określa sekcja 8.5.

3.5 Podział odpowiedzialności zespołu

Osoba	Obszar odpowiedzialności
Jan Zaborski	WAL, segmenty, manifest, indeksy dyskowe, checkpoint i recovery.
Hubert Błonowski	Parser, wykonawca zapytań, adapter HTTP, integracja i dashboard Grafany.
Obaj autorzy	Model danych, bufor i granice jego interfejsu, przegląd kontraktów, testy integracyjne, pomiary i dokumentacja swoich podsystemów.

Wykonawcę zapytań można rozwijać z pamięciową implementacją interfejsu odczytu. Magazyn można testować bez HTTP i parsera. Adapter HTTP korzysta z ustalonych typów wyników i błędów. Wspólna akceptacja zmian modelu i kontraktów poprzedza ich wdrażanie po obu stronach.

4 Zapis, buforowanie i obsługa opóźnionych danych

4.1 Algorytm przyjęcia paczki

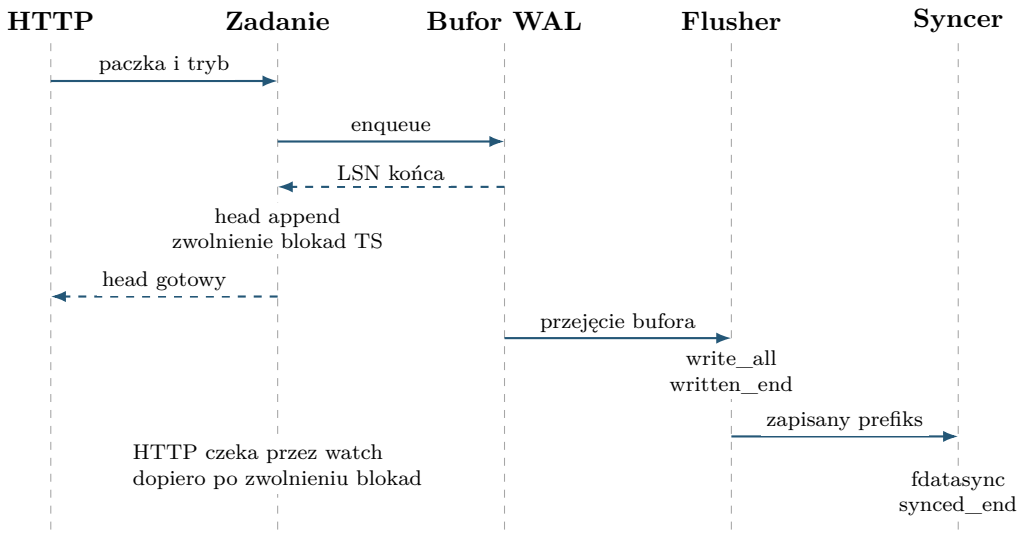
1. Adapter rezerwuje miejsce przyjęcia, sprawdza rozmiar i tryb trwałości, dekoduje JSON oraz kanonizuje punkty.
2. Zadanie uzyskuje uchwyty i blokady według sekcji 2.3. Waliduje całą paczkę, w tym duplikaty, opóźnienie i granice zamknięcia. Rezerwuje pojemność head, indeksów, buforów kodowania i WAL, aby po enqueue nie odrzucać już operacji.
3. Jeśli potrzebny jest flush albo oczekiwanie na pamięć, zwalnia blokady TS, barierę i mutex rejestracji przed czekaniem. Po ponowieniu waliduje stan od początku. Paczka niemieszcząca się nawet w pustym budżecie daje 422; przejściowy brak miejsca po maksymalnie 1 s oczekiwania daje 503, zawsze przed enqueue.
4. Koduje rekord w buforze roboczym lokalnym dla wątku. Rekord zawiera definicje nowych szeregów, a następnie wszystkie nowe próbki paczki. Pod mutexem WAL kopiuje pełną ramkę do bufora i otrzymuje końcowy LSN. Nie czeka na write ani fdatasync.
5. Nadal pod blokadami TS publikuje wszystkie próbki i metadane. Nowe TS i indeksy stają się publiczne dopiero po enqueue definicji i inicjalizacji head. Ustawia `last_lsn` zmienionych TS.
6. Zwalnia wszystkie blokady i barierę oraz kończy część roboczą. None zwraca sukces; Written lub Synced czeka asynchronicznie na odpowiednią granicę WAL i dopiero wtedy zwraca sukces.

Kolejność enqueue jest globalna, lecz publikacje rozłącznych TS mogą skończyć się w innej kolejności. Część pamięciowa zachowuje atomowość paczki, a nie transakcję łączącą publikację z fizyczną synchronizacją. Błąd I/O w tle kończy proces. Odpowiedź None mogła już zostać wysłana; odpowiedź Written lub Synced wymaga osiągnięcia odpowiedniej granicy. Recovery odtworzy również kompletne ramki, dla których klient nie otrzymał odpowiedzi albo publikacja head nie zdążyła się zakończyć.

4.2 Warunki przyjmowania out-of-order

Dla szeregu s oznaczamy:

- H_s – największy czas przyjęty w tym szeregu, także na dysku;
- S_s – granicę zamknięcia: największy czas w zamrożonych lub opublikowanych danych szeregu; przed pierwszym zrzutem jest nieokreślona;
- W – skonfigurowane maksymalne opóźnienie.



Rysunek 4: Część pamięciowa żądania i niezależne zadania I/O. None kończy się po publikacji head, Written po write, Synced po sync.

Aby wynik walidacji nie zależał od kolejności JSON, wszystkie punkty paczki sprawdza się względem H_s sprzed tej paczki. Każdy jej nowy punkt musi spełniać:

$$t > S_s \quad \text{oraz} \quad t \geq H_s - W.$$

Warunek z S_s pomija się, gdy nie zamrożono jeszcze bloku szeregu. Warunek z H_s pomija się dla nowego szeregu. Po publikacji całej paczki H_s zwiększa się do maksimum starej wartości i jej nowych czasów. Porównania wykonuje się z kontrolą przepełnienia; ujemna dolna granica okna jest równoważna zeru dla dopuszczonej dziedziny czasu. Identyczny duplikat obecny w otwartym buforze jest rozpoznawany przed testem W i nie staje się nowym zapisem; konflikt wartości jest błędem. Warunek zamkniętego zakresu ma pierwszeństwo przed obsługą duplikatów.

Czas próbki nie może wyprzedzać zegara serwera o więcej niż 60 s. Chroni to H_s przed przypadkowym przesunięciem daleko w przyszłość. Zegar ścienny jest używany do tego testu i domyślnego czasu zapytania; czas przebywania w RAM oraz timeouty są mierzone zegarem monotonicznym.

Stan szeregu	Punkt	Wynik
$H = 12{:}10$, $S = 12{:}00$, $W = 5$ min	$t = 12{:}06$	Przyjęcie do otwartego bufora.
Ten sam stan	$t = 12{:}04$	Odrzucenie: przekroczone okno opóźnienia.
Po zamrożeniu: $S = 12{:}10$	$t = 12{:}09$	Odrzucenie: zakres zamknięty, nawet jeśli flush jeszcze trwa.
Po zamrożeniu: $S = 12{:}10$	$t = 12{:}11$	Przyjęcie do nowego aktywnego bloku.

Przy $W = 0$ paczka może wprowadzić kilka nowych czasów, ale żaden nie może być starszy od H_s sprzed paczki. Punkty wewnątrz jednej atomowej paczki nie ustalają kolejności przyjmowania między sobą. Standardowy profil demonstracji korzysta z $W = 5$ min.

4.3 Organizacja i limit pamięci

Bufor szeregu jest wektorem 16-bajtowych próbek posortowanych po czasie. Zapis rosnący dopisuje na końcu, a out-of-order korzysta z wyszukiwania binarnego i wstawienia. Odczyt zakresu wymaga

dwóch wyszukiwań binarnych oraz przejścia po wybranym wycinku. Koszt wstawiania starszych próbek jest liniowy względem liczby przesuwanych elementów; udział takich zapisów będzie osobnym parametrem eksperymentu.

Pojemność wektorów zwiększa się o wielokrotności 4096 próbek przez `try_reserve_exact`; rozlicza się rzeczywistą pojemność przydzieloną przez kontener, nie samą liczbę punktów. Zmniejszenie limitu pamięci poniżej minimalnej wymaganej alokacji jest błędem konfiguracji lub odrzuceniem paczki, a nie powodem nieskończonego ponawiania flush.

Limit bufora obejmuje aktywne i zamrożone próbki, bloki podtrzymywane przez istniejące widoki odczytu oraz pojemność zapasową. Współdzielony blok jest liczony jeden raz, aż do zwolnienia ostatniego uchwytu. Katalog, kolejka i zapytania mają osobne limity. Przed utrwaleniem paczki sprawdza się możliwość przydziału; pojemność może zostać zarezerwowana wcześniej, lecz nieudany zapis nie publikuje próbek. Przy wzroście wymagającym realokacji uwzględnia się przejściowe współistnienie starego i nowego bloku pamięci.

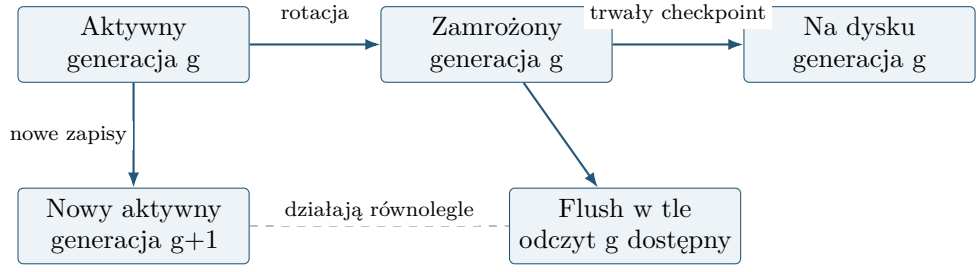
Zamrożenie przenosi własność wektora do `Arc<FrozenBlock>`, bez pełnej kopii generacji. Nowy blok aktywny alokuje pamięć w miarę napływu danych. Zapytania kopiują potrzebne wycinki tylko z bloków aktywnych; dla niezmiennych bloków przechowują uchwyty. Blok zamrożony może pozostać w RAM po publikacji segmentu, dopóki nie zakończy się wcześniejszy odczyt. To podtrzymanie pamięci jest uwzględniane w limicie i testach RSS.

4.4 Wyzwalanie i przebieg zrzutu

Timer sprawdzany co 1 s zleca zrzut, jeżeli najstarsza próbka bieżącej generacji przebywa w RAM przez skonfigurowany czas. Ten termin oznacza *rozpoczęcie* zrzutu; zwolnienie pamięci wymaga czasu I/O. Drugim wyzwalaczem jest brak miejsca na następną paczkę. Przy włączonym limicie zlecenie następuje już po przekroczeniu 80% budżetu, aby zostawić miejsce na nowe aktywne bloki. Pustej generacji nie zrzuca się.

Koordynator przeprowadza wspólne zamrożenie niepustych bloków head. Nie przełącza przy tym plików WAL. Po zwolnieniu bariery zapis do nowej generacji odbywa się równolegle z utrwalaniem poprzedniej. W tle działa jeden wykonawca flush; do jego zakończenia nie zamraża się następnej generacji. W razie upływu czasu buforowania żądanie kolejnej rotacji pozostaje oczekujące, ale nie tworzy nieograniczonej kolejki zamrożonych bloków.

Przy twardym limicie pamięci nowe paczki czekają krótko bez blokad lub otrzymują 503. Dostępność podczas flush nie oznacza nieograniczonego przyjmowania danych przy dowolnie wolnym dysku. Dokładną kolejność operacji i zakres usuwania WAL określa sekcja 6.



Rysunek 5: Rotacja tworzy nowy aktywny blok przed zapisem poprzedniego na dysk. Kolejna rotacja czeka na zakończenie poprzedniego flush.

5 Organizacja danych na dysku

5.1 Katalog i wspólne reguły formatu

```
data/
  LOCK
  MANIFEST
  wal/
    wal-00000000000000000001.log
    wal-00000000000000000002.log
  segments/
    segment-00000000000000000001.bin
    segment-00000000000000000002.bin
```

Numery segmentów danych identyfikują generacje head, a numery plików WAL niezależną kolejność plików dziennika. Obie numeracje mają po 20 pozycji. Pliki tymczasowe mają końcówkę `.tmp`. Zawsze istnieje jeden aktywny plik WAL, do którego wolno dopisywać, oraz ewentualne wcześniejsze pliki oczekujące na checkpoint. Manifest jest jedynym źródłem informacji o tym, które segmenty należą do bazy. Dla WAL recovery rozpoczyna od kursora zapisanego w manifeście i uwzględnia wszystkie późniejsze opublikowane pliki, także utworzone przed publikacją następnego checkpointu.

Wszystkie liczby binarne mają kolejność *little-endian*. Liczby zmiennoprzecinkowe zapisuje się przez 64-bitową reprezentację IEEE 754. Nie zapisuje się surowego układu struktur Rust ani paddingu. Napisy mają postać `u32byte_len` i dokładnie tylu bajtów UTF-8. Arytmetyka długości i offsetów jest sprawdzana przed alokacją i odczytem.

Wersja segmentów danych wynosi 2, a WAL i manifestu 3. Nowa wersja wprowadza bajtowy LSN i niezależne numeracje head i WAL. Nieznana wersja, niezerowe zastrzeżone flagi, nieprawidłowy UTF-8 i naruszenia zadeklarowanych długości są błędem uruchomienia. Zmiana wersji wymaga osobnej migracji; projekt nie zakłada zgodności plików obecnego prototypu z tą specyfikacją.

Sumy kontrolne to CRC-32/IEEE, zgodne z `crc32fast` [10]. Wartość testowa dla ASCII `123456789` wynosi `0xCBF43926`. CRC wykrywa przypadkowe uszkodzenia, ale nie zapewnia uwierzytelnienia. Sumy zapisuje się jako `u32` little-endian.

5.2 Bufor WAL, flusher i syncer

Usługa utrzymuje jeden logiczny strumień WAL. Bufor `Mutex<Vec<u8>>` przechowuje pełne ramki wraz z początkową pozycją logiczną. Żądanie koduje rekord w lokalnym buforze roboczym, a pod mutexem jedynie dopisuje bajty i zwiększa `enqueued_end` o długość ramki. Nie wykonuje operacji plikowych.

Trzy granice są monotonicznymi końcami ciągłych prefiksów:

$$0 \leq synced_end \leq written_end \leq enqueued_end.$$

Żądanie zapamiętuje własne `lsn=record_end`. Granice nie mówią, czy zadanie zdążyło już opublikować próbki w head. Dlatego checkpoint wymaga osobnej bariery części pamięciowej.

Tryb	Warunek odpowiedzi po publikacji head	Gwarancja
None	Bez oczekiwania na I/O.	Awaria procesu może utracić potwierdzoną paczkę; zachowane kompletne ramki nadal mogą zostać odtworzone.
Written	`written_end>=lsn`.	Cała ramka jest w systemie operacyjnym. Przy awarii samego procesu, bez awarii hosta i nośnika, potwierdzone dane są odtwarzane.

Tryb	Warunek odpowiedzi po publikacji head	Gwarancja
Synced	`synced_end>=lsn`.	Ramka objęta zakończonym fdatasync i trwała publikacja jej pliku. Obowiązują gwarancje synchronizacji lokalnego systemu plików.

Flusher jest jedynym wykonawcą dopisywania do plików WAL. Przy braku oczekujących klientów zapisuje bufor co najwyżej co 10 ms lub po zgromadzeniu 1 MiB. Written i Synced rejestrują cel oczekiwania i budzą flusher wcześniej. Przy takich celach flusher może odczekać do 200 mikrosekund od pobudzenia na zebranie grupy; jest to parametr do pomiarów, nie cel opóźnienia całego żądania.

Pod mutexem podmienia bufor na pusty i zachowuje zakres LSN przejętych bajtów. Po zwolnieniu mutexu zapisuje je sekwencyjnie przez `write_all`; pojedynczy `write` może zapisać mniej bajtów niż żądano [12]. Nowe żądania w tym czasie dopisują do nowego bufora. `written_end` zwiększa się dopiero po zapisaniu pełnych ramek, także dla porcji kończącej plik przed jego synchronizacją i rotacją. Flusher nie czeka na aktualizację head ani na blokady TS.

Syncer synchronizuje zapisane dane co 1 s albo wcześniej, gdy istnieje niezaspokojony cel Synced. Jeśli żądana ramka nadal jest w RAM, budzi flusher i czeka na odpowiedni postęp Written. Przed fdatasync zapamiętuje uchwyt do konkretnego pliku i granicę zapisanych bajtów. Po sukcesie potwierdza wyłącznie ten prefiks. Nie pobiera późniejszego written_end i nie uznaje dopisań wykonanych w trakcie synchronizacji za automatycznie trwałe.

Postęp obu granic jest publikowany przez wspólny kanał watch, z aktualizacją zachowującą monotoniczność obu pól. Oczekujący najpierw sprawdza bieżący stan, potem zmiany; nie polega na otrzymaniu każdego zdarzenia. Watch przechowuje ostatnią wartość, co odpowiada oczekiwaniu na przekroczenie granicy [15]. Zamknięcie kanału nie jest sukcesem. Timeout domyślnie 30 s lub anulowanie klienta kończy oczekiwanie bez wycofania danych.

Rotacja fizyczna następuje przed ramką, która przekroczyłaby 128 MiB pliku, razem z 32-bajtowym nagłówkiem. Rekordów nie dzieli się między pliki. Flusher synchronizuje stary plik, publikuje jego granicę Synced i tworzy nowy pod nazwą tymczasową. Zapisuje i synchronizuje nagłówek, wykonuje rename i synchronizuje katalog `wal/`; dopiero potem zapisuje ramki nowego pliku. Synchronizacja samego pliku nie zapewnia trwałości jego nowej nazwy w katalogu [13].

Flusher i syncer uzgadniają uchwyt pliku i jego granicę pod krótką blokadą stanu I/O, ale nie utrzymują jej przez fdatasync. Uchwyt pozostaje ważny do zakończenia operacji. Synced nowego pliku wymaga trwałego prefiksu wcześniejszych plików. Rotacja WAL nie zamraża head i nie wymaga bariery TS.

`sync_file_range` nie jest elementem protokołu trwałości. Może zostać dodane jako osobno zmierzona optymalizacja writeback, ale nie zastępuje fdatasync [14]. Okres 1 s nie ogranicza sam z siebie możliwej straty danych None: zależy ona również od opóźnienia flushera i ilości danych w RAM.

Budżet WAL domyślnie wynosi 32 MiB i obejmuje rzeczywistą pojemność bufora przyjmującego oraz bufora przejętego przez flusher. Bufory kodowania żądań należą do osobnego budżetu przyjęcia. Zapas lokalnych buforów roboczych pozostaje rozliczony także między żądaniami, dopóki ich pamięć nie zostanie zwolniona. Brak miejsca powoduje oczekiwanie bez blokad TS albo 503 przed enqueue. Nie dodaje się nieograniczonej kolejki pomiędzy buforem a flusherem.

Błąd write, fdatasync albo publikacji pliku WAL powoduje raport błędu i natychmiastowe zakończenie procesu. Nie wysyła się sukcesu celom, których granica nie została osiągnięta. None mogło już otrzymać odpowiedź. Recovery odtwarza poprawny prefiks; błąd nośnika lub pełnej ramki CRC może zatrzymać start, więc zakończenie procesu nie oznacza automatycznej naprawy dowolnej

usterki.

5.3 Format dziennika i bajtowy LSN

Plik ma następujący nagłówek:

Offset	Rozmiar	Pole	Znaczenie
0	8 B	magic	ASCII `RSTWAL03`.
8	2 B	version	`u16=3`.
10	2 B	flags	`u16=0`.
12	4 B	header_len	`u32=32`.
16	8 B	segment_id	`u64`; numer pliku zgodny z nazwą.
24	8 B	base_lsn	Logiczna pozycja początku pierwszej ramki.
32	zmienny	frames	Pełne ramki w kolejności enqueue.

LSN liczy wyłącznie bajty ramek, bez nagłówków plików. Koniec oznacza pozycję pierwszego bajtu po rekordzie, nie ostatniego bajtu należącego do rekordu. Dla końca rekordu na fizycznej pozycji p:

$$record_end = base_lsn + (p - 32).$$

W pustej bazie base_lsn wynosi 0. Przy rotacji następny plik ma base_lsn równy końcowi poprzedniego strumienia, a nie ten koniec plus 1. Przyrost LSN jest długością rekordu. Sumy i liczniki są sprawdzane przed przepełnieniem.

Ramka ma układ `len|crc32|kind|flags|reserved|payload`:

Offset	Rozmiar	Pole	Znaczenie
0	4 B	record_len	Cała długość ramki, od 20 B do 8 MiB.
4	4 B	crc32	CRC bajtów 0–3 i 8–końca ramki, z pominięciem pola CRC.
8	1 B	kind	`u8=1`, atomowa paczka ingest.
9	1 B	flags	`u8=0`.
10	2 B	reserved	`u16=0`.
12	record_len −12	payload	Definicje szeregów, a następnie próbki.

Payload zapisuje się w kolejności:

1. `new_series_count:u32`.
2. Dla każdej definicji: `id:u64`, napis metryki, `label_count:u16` i pary napisów nazwa–wartość. Definicje są uporządkowane po ID, etykiety kanonicznie.
3. `point_count:u32`, dodatnia liczba nowych próbek.
4. Próbki po 24 B: `series_id:u64`, `timestamp:i64`, `value_bits:u64`, uporządkowane po ID i czasie.

Duplikat bez zmian nie pojawia się w payloadzie. Każda nowa definicja ma co najmniej jedną próbkę w tej samej ramce. Identyfikator próbki musi pochodzić z checkpointu, wcześniejszej ramki

lub bieżącej definicji. Nieznanego ID nie odkłada się do przyszłego rekordu ani nie pomija. Brak definicji jest błędem formatu lub protokołu rejestracji.

Jedna próbka istniejącego szeregu daje payload $4 + 4 + 24 = 32$ B i ramkę $12 + 32 = 44$ B. Jeżeli jest pierwszą w pustej bazie, jej LSN końcowy wynosi 44, a fizyczny koniec pliku 76. Pełna ramka z poprawną sumą jest jednostką replay; nie ma osobnego rekordu commit ani gwarancji wycofania paczki bez odpowiedzi. Nowy typ kind lub niezerowe flagi wymagają rozszerzenia formatu.

5.4 Segment danych

Jeden flush tworzy jeden segment obejmujący wszystkie niepuste szeregi zamrożonej generacji head. Jego numer jest niezależny od plików WAL. W segmencie znajdują się: 48-bajtowy nagłówek, bloki próbek, tablica indeksu oraz 4-bajtowa suma całego pliku bez tej końcowej sumy.

Offset	Rozmiar	Pole	Znaczenie
0	8 B	magic	ASCII `RSTSEG02`.
8	2 B	version	`u16=2`.
10	2 B	flags	`u16=0`.
12	4 B	header_len	`u32=48`.
16	8 B	generation	Numer generacji `u64`.
24	8 B	cutoff_lsn	Końcowy bajtowy LSN prefiksu objętego zrzutem.
32	8 B	series_count	Liczba bloków i wpisów indeksu.
40	8 B	index_offset	Początek tablicy indeksu od początku pliku.

Bloki następują bez przerw, w rosnącej kolejności `SeriesId`. Blok zawiera wyłącznie pary `time stamp:i64`, `value_bits:u64`. Próbki są ściśle rosnące w czasie. Identyfikator szeregu nie jest powtarzany przy każdym punkcie. Pustych bloków nie zapisuje się.

Wpis indeksu zajmuje 48 B:

Offset	Rozmiar	Pole	Znaczenie
0	8 B	series_id	Identyfikator szeregu.
8	8 B	min_time	Pierwszy czas w bloku, `i64`.
16	8 B	max_time	Ostatni czas w bloku, `i64`.
24	8 B	data_offset	Początek bloku względem początku pliku.
32	8 B	point_count	Liczba próbek w bloku.
40	4 B	block_crc32	CRC surowych bajtów całego bloku.
44	4 B	reserved	Zero; miejsce zastrzeżone.

Dla N punktów i K niepustych szeregów:

$$\text{index_offset} = 48 + 16N, \qquad \text{file_size} = 48 + 16N + 48K + 4.$$

Segment jednego szeregu z dwiema próbkami ma więc 132 B, offset indeksu 80 i offset danych 48. Jego wpis indeksu zajmuje bajty 80–127, a końcowe CRC bajty 128–131.

Przy uruchomieniu kontroluje się długość, sumę całego segmentu, zakresy offsetów, kolejność identyfikatorów, kolejność czasów i zgodność min/max. Indeks pozostaje w RAM; próbki są

odczytywane na żądanie. Przy odczycie całego bloku można dodatkowo sprawdzić jego CRC. Odczyt wycinka korzysta z pozycyjnego odczytu i wyszukiwania binarnego; nie wymaga wczytania całego szeregu. Podczas pracy żaden inny proces nie może modyfikować plików.

5.5 Manifest i katalog szeregów

Plik `MANIFEST` jest kopertą binarną z dokumentem JSON UTF-8. Nagłówek ma 16 B: magic `RSTMAN03` (8 B), `version:u16=3`, `flags:u16=0`, `json_len:u32`. Dalej znajdują się bajty JSON i CRC-32 całego nagłówka oraz payloadu. Maksymalna długość JSON to 64 MiB; dodatkowe bajty po CRC są błędem.

Przykład dotyczy pierwszej paczki dwóch punktów z API (sekcja 8): jej ramka ma 136 B, a koniec w pliku WAL z 32-bajtowym nagłówkiem wypada na pozycji 168. Wartość CRC segmentu jest miejscem na sumę wyliczaną z rzeczywistych danych.

```
{
  "format_version": 3,
  "generation": 1,
  "checkpoint_lsn": 136,
  "replay": {"segment_id": 1, "offset": 168},
  "next_series_id": 2,
  "segments": [
    {
      "generation": 1,
      "file": "segment-00000000000000000001.bin",
      "cutoff_lsn": 136,
      "size_bytes": 132,
      "crc32": 0
    }
  ],
  "series": [
    {
      "id": 1,
      "metric": "temperature_celsius",
      "labels": [["instance", "1"], ["service", "api"]],
      "sealed_through": 1792843201000
    }
  ]
}
```

`generation` jest numerem zrzutu head, nie numerem pliku WAL. `checkpoint_lsn` jest logicznym końcem wszystkich rekordów odwzorowanych w opublikowanych segmentach. `replay` wskazuje fizyczny koniec ostatniej ramki objętej checkpointem. Plik zawierający tę pozycję pozostaje kotwicą recovery nawet wtedy, gdy kończy się dokładnie na tej granicy. Kursor musi spełniać

$$checkpoint_lsn = base_lsn + (replay.offset - 32)$$

i wskazywać granicę pełnego rekordu lub początek pustego strumienia.

Manifest zawiera cały katalog i granice zamknięcia z chwili zamrożenia head. Nie dołącza późniejszych szeregów ani stanów aktywnych bloków; ich definicje i próbki znajdują się w późniejszych rekordach WAL. Tablice szeregów i segmentów są uporządkowane rosnąco po ID i generacji. Nazwy segmentów są pojedynczymi komponentami ścieżki bez możliwości wyjścia z katalogu. CRC wpisu segmentu powtarza końcowe CRC pliku. JSON nie wymaga konkretnej kolejności pól; sumę liczy się z zapisanych bajtów, a liczniki dekoduje jako liczby całkowite, bez pośredniego f64.

Początkowy manifest ma generację i checkpoint równe 0, `replay:{"segment_id":1,"offset":32}`, `next_series_id=1` i puste tablice. Nie przechowuje numeru następnej ramki: następny LSN wynika z końca odtworzonego strumienia bajtów. Po replay licznik ID przekracza wszystkie identyfikatory z katalogu i późniejszych rekordów. GC i aliasowanie ID nie należą do MVP.

Walidacja sprawdza wersję, CRC, kursor, rozmiary i sumy segmentów, rosnące generacje zrzutów, brak powtórzonych identyfikatorów i rozłączność zakresów bloków danego szeregu. `sealed_through` odpowiada największemu czasowi szeregu w opublikowanych segmentach. Każdy szereg katalogu ma co najmniej jeden blok; przed replay odtwarza się $H_s = S_s$.

5.6 Indeksy w pamięci i plan odczytu

TSMap korzysta z mapy `SeriesKey->Arc<TS>` i wektora uchwytów TS. Indeks odwrócony przechowuje uporządkowane identyfikatory dla nazwy metryki i każdej pary etykiety. Wybór szeregu zaczyna się od listy metryki, następnie przecina listy wymaganych etykiet i sprawdza pełny klucz. Matcher z pustą wartością wybiera brak etykiety; nie wymaga przechowywania pustych etykiet w indeksie.

W stanie każdego TS znajdują się uporządkowane deskryptory bloków: segment, min/max, offset i liczba próbek. Z reguły $t > S_s$ wynika, że zakresy kolejnych bloków tego samego szeregu nie nakładają się. Przeszukanie deskryptorów eliminuje segmenty spoza zakresu, a wyszukiwanie binarne wybiera próbki w pozostałych blokach.

Odtworzenie indeksów nie wymaga relacyjnej bazy ani odrębnego serwera. Koszt pamięci katalogu zależy od liczby szeregów, długości etykiet i liczby segmentów, nie tylko od liczby próbek. Brak kompaktowania oznacza narastanie liczby deskryptorów przy długim działaniu.

6 Checkpoint i odzyskiwanie po awarii

6.1 Trwałe opublikowanie generacji

Generacja head g i numer pliku WAL są niezależne. Checkpoint powstaje przez krótkie zamrożenie head i utrwalenie w tle. W jednym momencie działa najwyżej jeden flush danych.

Bariera zamrożenia head:

1. Wstrzymać rozpoczynanie części pamięciowych zapisów i uzyskać wyłączną barierę. Poczekać, aż rozpoczęte zadania zakończą enqueue i head append. Klienci oczekujący na Written lub Synced nie utrzymują bariery i nie opóźniają tego kroku.
2. Odczytać $C = enqueued_end$. Wszystkie rekordy do C są już zastosowane w head. Samo written_end albo synced_end nie wystarczyłoby: rekord może trafić na dysk przed zakończeniem publikacji jego TS.
3. Zapamiętać katalog i następny ID. Pod blokadami TS przenieść aktywne wektory do zamrożonych bloków generacji g, zaktualizować S_s i zainicjalizować aktywne bloki $g + 1$.
4. Zwolnić barierę. Nowe żądania przyjmują pozycje WAL większe od C i zapisują do nowych aktywnych bloków. Nie rotować pliku WAL ani nie wykonywać synchronizacji w tej sekcji.

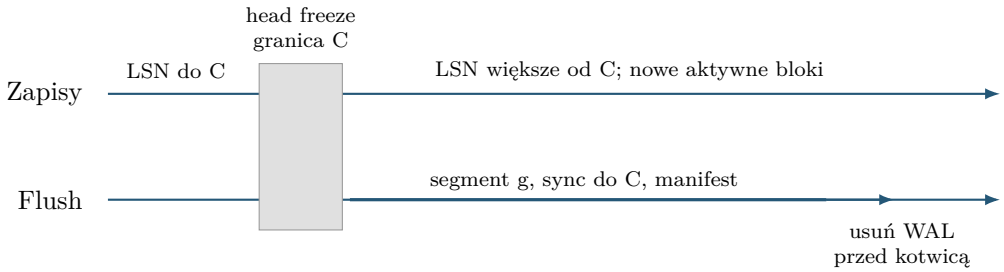
Utrwalanie poza barierą i blokadami TS:

1. Zapisać `segments/segment-<g>.bin.tmp` z cutoff C, zsynchronizować plik, zmienić nazwę na docelową i zsynchronizować katalog `segments/`.
2. Wywołać `wal.wait_for(C,Synced)`. To oczekiwanie dotyczy zadania checkpointu, nie blokuje TS. Zapewnia trwałą obecność pozycji wznowienia WAL, także gdy część paczek miała tryb None. Odszukać plik a zawierający ostatnią ramkę do C i jej fizyczny koniec p.
3. Zapisać i zsynchronizować `MANIFEST.tmp` ze spisem segmentów, katalogiem z chwili zamrożenia, generacją g, checkpointem C i kursorem `replay=(a,p)`. Atomowo zastąpić manifest i zsynchronizować katalog danych. Dopiero wtedy checkpoint jest opublikowany.

4. Pod krótką blokadą każdego TS podmienić zamrożony blok g na deskryptor dyskowy. Nie nadpisywać aktywnych próbek, H_s ani last_lsn z nowszych żądań. Istniejące ReadView zachowują swoje uchwyty.

5. Usunąć wyłącznie zamknięte WAL o numerach mniejszych od a, których ostatni LSN jest $\leq C$. Zachować plik-kotwicę a, wszystkie późniejsze i plik aktywny. Flusher potwierdza status zamknięcia; nie usuwa się pliku, do którego może jeszcze dopisać. Zsynchronizować katalog WAL, zwolnić własne uchwyty RAM i zakończyć flush.

Zachowanie kotwicy upraszcza replay: nawet gdy p wypada na końcu pliku, pozostaje jednoznaczna pozycja wznowienia. Kosztem jest pozostawienie jednego dodatkowego pliku do następnego checkpointu. Nie trzeba kopiować ani obcinać początku aktywnego WAL.



Rysunek 6: Bariera dotyczy tylko stanu head. Rotacja WAL i synchronizacja trwałości nie są częścią tej pauzy.

System docelowy to Linux z lokalnym systemem plików obsługującym atomowy rename oraz synchronizację plików i katalogów. Written nie gwarantuje przetrwania awarii hosta. Synced oraz checkpoint opierają się na poprawnych gwarancjach fdatasync/fsync; nie obejmują fizycznego uszkodzenia nośnika.

6.2 Algorytm uruchomienia

1. Sprawdzić konfigurację i uzyskać wyłączną blokadę katalogu. Gotowość pozostaje fałszywa do zakończenia recovery.

2. W pustym katalogu utworzyć i trwale opublikować WAL nr 1 z base_lsn równym 0, potem początkowy manifest z kursorem (1,32). Dokończenie przerwanej inicjalizacji jest dozwolone tylko dla poprawnego pustego WAL i braku danych oraz opublikowanych segmentów. Inny brak manifestu jest błędem, nie resetem bazy.

3. Zweryfikować manifest i wskazane segmenty; odbudować TSMap i indeksy. Odtworzyć granice zamknięcia oraz katalog ID. Istniejące TS otrzymują początkowe last_lsn równe checkpointowi; późniejsze rekordy replay aktualizują je własnym LSN. Nie dołączać automatycznie segmentów danych spoza manifestu.

4. Odczytać kursor `replay=(a,p)` i checkpoint C. Plik a musi istnieć; sprawdzić nagłówek, zakres p i jego zgodność z bajtowym LSN C. Zebrać wszystkie opublikowane WAL od a do najwyższego h, bez luk w numerach.

5. Od pozycji p pliku a czytać kolejne pełne rekordy, potem następne pliki od ich nagłówków. Sprawdzać długości, kind, flagi, CRC i ciągłość pozycji: base_lsn następnego pliku odpowiada końcowi logicznemu poprzedniego. Stare bajty przed kursorem są pokryte checkpointem i nie są replayowane.

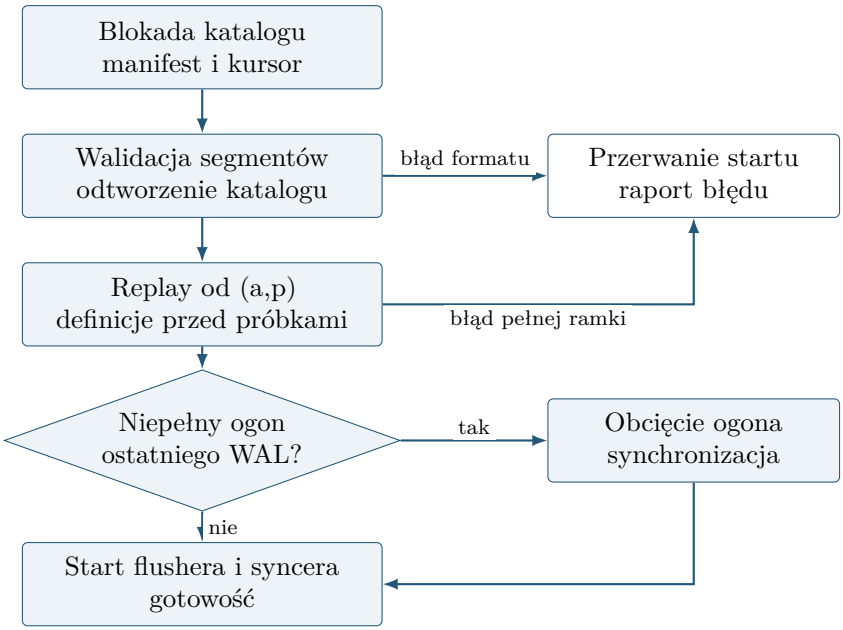
6. Każdy rekord zastosować w całości: najpierw definicje szeregów, potem próbki. Brak definicji lub ponowne zdefiniowanie klucza pod innym ID jest błędem. Nie ma GC, aliasowania ID ani cichego pomijania. Po replay licznik ID przekracza wszystkie zaobserwowane identyfikatory.

7. Niepełny końcowy nagłówek ramki lub payload można odciąć wyłącznie w ostatnim WAL h. Pełna ramka z błędnym CRC, nieznany kind, luka lub uszkodzenie wcześniejszego pliku zatrzymuje start.

8. Zsynchronizować naprawiony i odtworzony aktywny plik przed ogłoszeniem gotowości. Ustawić enqueued_end, written_end i synced_end na koniec odtworzonego strumienia; starsze zamknięte pliki były synchronizowane przy rotacji. Otworzyć wyłącznie WAL h do dopisywania, a odtworzone próbki umieścić w aktywnych blokach generacji head $g + 1$.

9. Po udanej walidacji usunąć pliki tymczasowe, nieopublikowane segmenty danych i zbędne WAL sprzed kotwicy. Zachować wszystkie pliki od a wzwyż, dopóki kolejny checkpoint nie przesunie kursora. Uruchomić flusher i syncer, ogłosić gotowość.

Replay nie stosuje ponownie walidacji względem aktualnego zegara, zmienionego okna out-of-order ani limitu dopuszczalnego opóźnienia. Nie odrzuca w ten sposób wcześniejszych zapisów. Pierwotny czas przebywania w RAM nie jest utrwalany; odzyskane próbki kwalifikują się od razu do zrzutu czasowego. Za mały nowy limit pamięci zatrzymuje start z opisem problemu.



Rysunek 7: Replay rozpoczyna się od trwałego kursora checkpointu. Uszkodzona pełna ramka nie jest traktowana jak niepełny ogon.

6.3 Scenariusze awarii i granice gwarancji

Moment awarii	Stan po restarcie	Znaczenie dla klienta
Przed enqueue	Brak nowej ramki.	Brak sukcesu żądania.
Po enqueue, przed head append	Ramka może pozostać tylko w RAM albo być już zapisana.	Bez odpowiedzi; replay może zastosować całość.
Po odpowiedzi None, przed write	Paczka może zniknąć.	To dopuszczona słabsza trwałość, nie błąd recovery.
Po Written, przed Synced	Przy awarii procesu ramka wraca z WAL.	Awaria hosta może utracić niesynchronizowane dane.

Moment awarii	Stan po restarcie	Znaczenie dla klienta
Po Synced	Ramka pochodzi z WAL lub checkpointu.	Potwierdzone dane przetrwają w granicach gwarancji nośnika i systemu plików.
W środku końcowej ramki	Niepełny ogon ostatniego pliku jest odcięty.	Brak potwierdzenia Written/Synced dla tej ramki.
Podczas zapisu segmentu danych	Obowiązuje stary manifest i jego kursor.	Replay obejmuje także zapisy przyjęte podczas flush.
Po rename segmentu, przed manifestem	Segment jest sierotą; stary checkpoint pozostaje autorytatywny.	Dane nie są odtwarzane podwójnie.
Po manifest, przed usunięciem WAL	Replay zaczyna się od nowego kursora.	Stare pliki nie są ponownie stosowane.
Podczas usuwania starych WAL	Kotwica i późniejsze pliki pozostają.	Zapisy po C nadal można odtworzyć.

Błąd zapisu segmentu danych zatrzymuje nowe przyjęcia, zachowując head i WAL do diagnostyki lub restartu. Błąd I/O samego WAL jest polityką fail-stop i kończy proces. Granice zamknięcia z nieopublikowanego zrzutu mogą po restarcie wrócić do poprzedniego checkpointu; nie narusza to gwarancji Written/Synced.

6.4 Cykl życia procesu

Proces przechodzi przez inicjalizację, recovery, gotowość i zamykanie. Błąd flush danych może pozostawić proces z fałszywą gotowością i dostępnym odczytem; błąd I/O WAL kończy działanie.

Po SIGTERM serwer zamyka przyjęcia, kończy części pamięciowe zadań, wyznacza końcowy enqueued_end i wymusza zapis oraz synchronizację całego bufora WAL, także rekordów None. Dopiero potem zatrzymuje flusher i syncer. Kończy albo bezpiecznie przerywa zadanie flush danych bez usuwania plików potrzebnych do replay. Pełny flush head nie jest wymagany. Limit 30 s może zakończyć się wymuszonym zatrzymaniem; wówczas obowiązują zwykłe gwarancje wybranych poziomów trwałości.

7 Język i wykonywanie zapytań

7.1 Obsługiwana składnia

Podstawowy podzbiór obejmuje nazwany selektor z dokładnymi matcherami oraz pojedynczą funkcję agregującą próbki jednego szeregu w czasie. W celu obsługi prostych prób połączenia źródła danych dodaje się wyrażenie skalarne, np. `1+1`. Nie rozszerza to zakresu operacji między szeregami. Docelowa gramatyka ma postać:

```
expr       = selector | function "(" selector "[" duration "]" ")"
           | scalar ;
selector   = metric [ "{" [ matcher { "," matcher } ] "}" ] ;
matcher    = label "=" string ;
function   = "avg_over_time" | "min_over_time" | "max_over_time"
           | "sum_over_time" | "count_over_time" ;
duration   = positive_integer unit ;
unit       = "ms" | "s" | "m" | "h" | "d" | "w" ;
scalar     = number [ "+" number ] ;
```

Nazwy mają składnię z sekcji 3. Napisy w podwójnych cudzysłowach korzystają z escape'ów JSON. Liczba skalarna ma składnię liczby JSON i musi reprezentować skończone `f64`. Białe znaki między tokenami są ignorowane. Parser musi zużyć całe wejście. Powtórzony matcher tej samej etykiety jest błędem. Nazw etykiet z prefiksem `__` nie dopuszcza się w matcherach tej wersji. Nazwę metryki podaje się bezpośrednio.

Przykłady:

```
temperature_celsius
temperature_celsius{service="api",instance="1"}
avg_over_time(temperature_celsius{service="api"}[5m])
count_over_time(temperature_celsius[30s])
1+1
```

Odrzucane są regexy, matchery inne niż `=`, selektory bez nazwy metryki, samodzielne wektory zakresowe, podzapytania, `offset`, `@`, zagnieżdżone funkcje i agregacje między szeregami. Przykładowo `sum(temperature_celsius)` nie oznacza sumy próbek w czasie i kończy się błędem nieobsługiwanej konstrukcji. Maksymalna długość tekstu zapytania wynosi 16 KiB.

Parser tworzy AST przez analizę tokenów i rekursywny zstępujący parser tej niewielkiej gramatyki. Nie jest potrzebny generator pełnego PromQL. Walidacja semantyczna sprawdza nazwy, dodatnie i mieszczące się w zakresie okno oraz obsługiwany typ wyniku. Błąd zawiera offset bajtowy wejścia i czytelny opis.

7.2 Czas obliczenia i wybór próbek

Zapytanie chwilowe oblicza wyrażenie dla jednego czasu T. Zakresowe oblicza je dla:

$$T_k = start + k \cdot step, \qquad k = 0, \ldots, \left\lfloor \frac{end - start}{step} \right\rfloor.$$

Krok jest dodatni, $start \le end$; koniec jest uwzględniony tylko wtedy, gdy wypada na siatce. Obliczenia używają całkowitych milisekund i sprawdzanej arytmetyki.

Dla samego selektora wybiera się najnowszą próbkę o czasie $t \le T$ oraz $t \ge T - D$, gdzie D to lookback, domyślnie 5 min. Jeżeli jej nie ma, szereg nie wnosi elementu wyniku. To wybór ostatniego pomiaru, nie interpolacja ani średnia. Timestamp w odpowiedzi wektora jest czasem obliczenia T, a nie oryginalnym czasem próbki. Nie implementuje się specjalnych znaczników staleness.

Funkcja z oknem R bierze próbki z przedziału

$$(T - R,\ T].$$

Próbka na lewej granicy jest wykluczona, na prawej włączona. Takie granice i powtarzanie obliczenia w krokach są zgodne z modelem zapytań Prometheusa [4]. Lookback selektora nie ogranicza okna funkcji. Dla każdego pasującego szeregu oblicza się osobny wynik.

Funkcja	Wynik dla niepustego zbioru $x_1, \ldots, x_n$
`avg_over_time`	Średnia arytmetyczna; każda próbka ma tę samą wagę.
`min_over_time`	Najmniejsza wartość.
`max_over_time`	Największa wartość.
`sum_over_time`	Suma wartości w rosnącym porządku czasu.
`count_over_time`	Liczba próbek, zwracana jako wartość liczbowa.

Dla pustego okna wszystkie pięć funkcji pomija szereg, również `count_over_time`; brak danych nie jest zerem. Agregacje zachowują etykiety źródłowego szeregu, ale usuwają nazwę metryki z zestawu etykiet wyniku. Selektor zachowuje `__name__`. Definicje funkcji są ograniczone do próbek liczbowych

[5]. Obliczenia korzystają z `f64` i sumowania kompensowanego. Średnia wynika z sumy i liczności; jeżeli suma się przepełnia, średnią przelicza się ze składników x_i/n, aby uniknąć przepełnienia samej sumy przy skończonej średniej. Wynik nieskończony lub NaN po przepełnieniu jest kodowany jako napis, a nie niepoprawna liczba JSON.

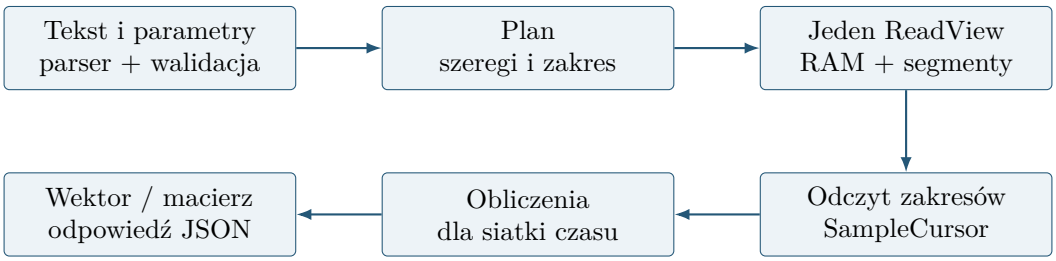
7.3 Przykład rozstrzygający granice

Niech szereg ma punkty $(0\,\mathrm{s}, 10)$, $(60\,\mathrm{s}, 20)$, $(120\,\mathrm{s}, 30)$ i $(180\,\mathrm{s}, 40)$. Dla `avg_over_time(metric[2m])`:

- w chwili 120 s okno $(0, 120]$ obejmuje 20 i 30, wynik wynosi 25;
- w chwili 180 s okno $(60, 180]$ obejmuje 30 i 40, wynik wynosi 35;
- w chwili 300 s okno $(180, 300]$ jest puste, więc nie ma elementu.

Zapytanie zakresowe od 120 s do 190 s z krokiem 60 s oblicza wyrażenie tylko w 120 s i 180 s. Wynik nie zmienia się, jeśli część powyższych próbek przeniesiono do segmentu.

7.4 Plan wykonania i limity pracy



Rysunek 8: Przepływ zapytania. Każdy etap może zakończyć się błędem lub przekroczeniem budżetu.

Plan wyznacza potrzebny przedział $[start - D, end]$ dla selektora albo $(start - R, end]$ dla funkcji. Granice są przycinane do dziedziny czasu bez przepełnienia. Potrzebne punkty aktywnych bloków są kopiowane jeden raz podczas tworzenia widoku; zamrożone bloki są współdzielone. Segmenty odczytywane są blokowo z buforem I/O 64 KiB. Agregacje korzystają z przesuwającego się okna: sumy i liczniki utrzymują stan, a minimum i maksimum kolejki monotoniczne. Nie odczytuje się tego samego całego zakresu od nowa dla każdego kroku. Po utracie skończoności stanu sumy odtwarza się go z próbek bieżącego okna, aby opuszczenie okna przez skrajną wartość nie zatruwało wszystkich następnych wyników.

Limity pojedynczego zapytania wynoszą: 1000 szeregów, 2 mln różnych próbek wejściowych, 1 mln punktów wyniku łącznie, 64 MiB pamięci roboczej i 30 s czasu. Metadane widoku, własne kopie aktywnych próbek, stan okien i wynik przed serializacją są rozliczane razem. Jeżeli limit nie wystarcza, zapytanie zwraca błąd bez częściowego wyniku. Dane na dysku można czytać strumieniowo, ale nadal obowiązuje limit próbek. Timeout jest sprawdzany również w pętlach obliczeń i odczytu; przerwanie klienta sygnalizuje anulowanie. Współdzielona pamięć zamrożonych bloków należy do globalnego budżetu bufora, także gdy podtrzymuje ją wyłącznie istniejący ReadView.

Szeregi wyniku porządkuje się leksykograficznie po kluczu kanonicznym, a punkty wewnątrz szeregu chronologicznie. API zapisuje wartości `f64` jako napisy, timestampy zaś jako liczby sekund Unix. Wynik skalarny ma format pary czasu i wartości; dla zapytania zakresowego jest macierzą z jednym szeregiem o pustych etykietach.

8 Komunikacja i publiczne API

8.1 Transport i konwencje

Serwer udostępnia HTTP/1.1, kodowanie UTF-8 i JSON. Nie wymaga WebSocketów ani brokera wiadomości. Połączenia mogą być utrzymywane przez keep-alive. Żądania zapisu używają `application/json`; odpowiedzi JSON mają ten sam typ. Nie implementuje się odbierania skomprymowanych żądań.

API zapytań wykorzystuje nazwy endpointów, kopertę odpowiedzi oraz wektory i macierze Prometheusa [3]. Obsługa PromQL i endpointów jest ograniczona do poniższego profilu; nie deklaruje się pełnej implementacji serwera Prometheus. Własny endpoint zapisu nie jest Prometheus Remote Write ani formatem OpenMetrics.

8.2 Zapis pojedynczy i paczkowy

`POST/api/v1/write` przyjmuje dokładnie jeden z dwóch wariantów: obiekt pojedynczego pomiaru albo obiekt zawierający wyłącznie niepustą tablicę `points`. Każdy pomiar wymaga pól `metric`, `timestamp` i `value`; brak `labels` oznacza pusty zbiór. `timestamp` jest całkowitą liczbą **milisekund**, a `value` liczbą JSON. W jednej paczce wolno przesłać do 10 000 punktów. Maksymalny rozmiar surowego żądania wynosi 4 MiB. Opcjonalny parametr query `durability=none|written|synced` nadpisuje ustawienie procesu; domyślnie jest to written. Powtórzony parametr, nieznana nazwa albo wartość jest błędem 400. Tryb dotyczy całej paczki i nie zmienia jej atomowości.

```
POST /api/v1/write
Content-Type: application/json

{
  "metric": "temperature_celsius",
  "labels": {"service": "api", "instance": "1"},
  "timestamp": 1792843200000,
  "value": 24.5
}
```

Wariant paczkowy:

```
{
  "points": [
    {
      "metric": "temperature_celsius",
      "labels": {"service": "api", "instance": "1"},
      "timestamp": 1792843200000,
      "value": 24.5
    },
    {
      "metric": "temperature_celsius",
      "labels": {"service": "api", "instance": "1"},
      "timestamp": 1792843201000,
      "value": 25.5
    }
  ]
}
```

Odpowiedź dla pierwszej paczki dwóch nowych punktów w pustej bazie:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
```

```
  "status": "success",
  "data": {
    "inserted": 2, "duplicates": 0,
    "lsn": 136, "durability": "written"
  }
}
```

`inserted+duplicates` jest liczbą punktów wejściowych. `lsn` jest końcem ramki, nie kolejnym numerem żądania. `durability` potwierdza wybrany i osiągnięty poziom. HTTP 200 przy None nie oznacza zapisu na dysk. Wszystkie tryby zapewniają widoczność całej paczki przed odpowiedzią.

Konflikt, błędne dane i brak budżetu odrzucają całą paczkę przed enqueue. Po nim zerwanie połączenia, anulowanie lub timeout nie wycofuje head. Timeout oczekiwania daje 503 z `errorType: "timeout"` i oznacza niepewny wynik z punktu widzenia klienta. Rekord może osiągnąć Written lub Synced już po tej odpowiedzi. Dla samych duplikatów żądanie nadal czeka na wymagany poziom dla zwróconego LSN; nie potwierdza silniejszej trwałości samą obecnością próbki w head.

8.3 Zapytania chwilowe i zakresowe

Obsługiwane są metody GET oraz POST z `application/x-www-form-urlencoded`. Nie wysyła się JSON jako ciała zapytania Prometheus. Parametry w danym żądaniu pochodzą albo z query string, albo z formularza; duplikaty parametrów i mieszanie obu źródeł są błędem. Całość parametrów nie może przekraczać 32 KiB.

Endpoint	Parametry	Znaczenie
`/api/v1/query`	`query`; opcjonalnie `time`, `timeout`	`time` domyślnie równe chwili odebrania żądania.
`/api/v1/query_range`	`query`, `start`, `end`, `step`; opcjonalnie `timeout`	Obliczenie na siatce czasu; wszystkie cztery główne parametry są wymagane.

Czasy przyjmują sekundy Unix z częścią ułamkową lub RFC 3339 ze strefą. Po konwersji muszą mieścić się w zakresie czasu z sekcji 3. Wymagana jest dokładność co najwyżej milisekundowa; dodatkowe niezerowe cyfry są odrzucane zamiast cichego zaokrąglania. Krok przyjmuje dodatnią liczbę sekund, np. `0.5`, albo pojedynczą jednostkę jak `15s`. Okno i timeout przyjmują dodatnią liczbę całkowitą z jedną jednostką z gramatyki `duration`. Timeout efektywny jest mniejszą wartością limitu serwera i parametru klienta.

Nieznane parametry zwracają 400; nie są ignorowane. Parametry opcjonalne pełnego API Prometheusa, np. `lookback_delta` i `limit`, pozostają nieobsługiwane w tym profilu. Grafana korzysta z konfiguracji nieprzesyłającej tych rozszerzeń.

```
GET /api/v1/query?query=temperature_celsius&time=1792843201

{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "__name__": "temperature_celsius",
          "service": "api",
          "instance": "1"
        },
```

```
        "value": [1792843201, "25.5"]
      }
    ]
  }
}
```

Przykład odpowiedzi zakresowej dla tej samej metryki od 1792843200 do 1792843201 z krokiem 1 s:

```
{
  "status": "success",
  "data": {
    "resultType": "matrix",
    "result": [
      {
        "metric": {
          "__name__": "temperature_celsius",
          "service": "api",
          "instance": "1"
        },
        "values": [[1792843200, "24.5"], [1792843201, "25.5"]]
      }
    ]
  }
}
```

Brak danych oznacza `result:[]` z HTTP 200. Wynik `1+1` w zapytaniu chwilowym ma `resultType: "scalar"` oraz `result:[1792843201,"2"]` dla powyższego czasu. Wartości specjalne wyników obliczeń koduje się jako `"NaN"`, `"+Inf"` lub `"-Inf"`. Wejście zapisu nadal ich nie dopuszcza.

8.4 Metadane i stan serwera

Metoda i ścieżka	Kontrakt profilu
GET `/api/v1/labels`	Posortowana lista nazw etykiet oraz `__name__` z katalogu.
GET `/api/v1/label/<name>/values`	Posortowane unikalne wartości etykiety; dla `__name__` nazwy metryk. Nieistniejąca poprawna nazwa daje pustą listę.
GET `/api/v1/status/buildinfo`	Koperta sukcesu z `data.version`, `data.revision`, `data.branch`, `data.buildDate` i `data.implementation`. To dane kompilacji TSDB; nie podszywają się pod wersję Prometheusa.
GET `/healthz`	200 i `ok` jako tekst, jeśli działa pętla obsługi procesu.
GET `/readyz`	200 i `ready` po recovery i przy sprawnym zapisie; 503 i tekst przy braku gotowości.

Odpowiedzi metadanych stosują kopertę `status:"success",data:[...]`. Endpointy etykiet nie przyjmują w tym profilu filtrów czasu ani `match[]`; operują na całym katalogu. Nie obsługuje się `/series`, `/metadata`, reguł, exemplarów ani administracyjnego usuwania danych. Dashboard nie wymaga tych operacji.

8.5 Błędy i przeciążenie

HTTP	errorType	Zastosowanie
400	`bad_data`	Błędny JSON, parametry, składnia, nazwy, typy albo nieobsługiwana konstrukcja języka.
404	`not_found`	Nieznany endpoint.
405	`bad_data`	Niewłaściwa metoda; nagłówek `Allow`.
409	`conflict`	Różne wartości dla tego samego czasu i szeregu.
413	`too_large`	Za duże ciało, tekst zapytania lub liczba punktów paczki.
415	`bad_data`	Nieobsługiwany typ treści lub kodowanie wejścia.
422	`execution`	Zamknięty zakres, przekroczone okno opóźnienia, limit szeregów, pamięci lub rozmiaru wykonywanego zapytania.
503	`unavailable` / `timeout`	Pełna kolejka, zajęte miejsca zapytań, brak gotowości, timeout albo zatrzymany zapis po błędzie I/O.
500	`internal`	Nieoczekiwany błąd lub uszkodzenie podczas odczytu; szczegóły ścieżek i systemu trafiają do logów.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "status": "error",
  "errorType": "execution",
  "error": "point belongs to a sealed time range",
  "details": {"reason": "sealed_range", "point_index": 1}
}
```

`details` jest rozszerzeniem własnego endpointu zapisu; indeks jest liczony od zera w oryginalnej paczce. Dla zapytań wystarcza standardowa koperta z `status`, `errorType` i `error`. Powody odrzucenia out-of-order rozróżnia się jako `sealed_range` i `out_of_order_window`.

Globalny semafor dopuszcza do 8 żądań zapisu łącznie: dekodowanych, oczekujących i przetwarzanych. Adapter rezerwuje miejsce przed zbuforowaniem ciała. Wewnątrz limitu pula wykonuje domyślnie cztery operacje równolegle; kolejka przekazuje zadania FIFO, ale nie narzuca kolejności zakończenia rozłącznych paczek. Zajęte miejsca skutkują od razu 503 z `Retry-After:1`. Oczekiwanie na pamięć nie utrzymuje blokad potrzebnych flush. Limit ośmiu przyjęć obejmuje również żądania oczekujące na Written/Synced, choć nie zajmują już one miejsca w puli części pamięciowej. Zapytania korzystają z osobnego semafora dwóch miejsc, również bez nieograniczonego oczekiwania.

Generator ponawia 503 z wykładniczym opóźnieniem od 1 s do 30 s i losowym odchyleniem. Nie ponawia automatycznie błędów walidacji, konfliktu i zamkniętego zakresu. Po błędzie połączenia dopuszcza niepewność wyniku opisaną wcześniej.

9 Technologie, konfiguracja i środowisko

9.1 Wybór technologii

Obszar	Wybór	Alternatywy	Uzasadnienie i ograniczenia
Język	Rust, edycja 2024	C++, Go	Kontrola reprezentacji danych i własności pamięci. Koszt nauki reguł własności; brak automatycznej ochrony przed nieograniczonymi alokacjami.
HTTP i runtime	Axum 0.8, Tokio 1	Actix Web, bezpośrednio Hyper	Routing, ekstraktory i integracja z Tokio [8]. Blokujące operacje plikowe trafiają poza wątki obsługi sieci.
Serializacja	Serde 1, serde_json 1	Ręczny parser JSON	Wspólne DTO i walidacja typów. Osobno wykrywamy duplikaty kluczy i sprawdzamy własne limity.
Czas i konfiguracja	time 0.3, Clap 4	chrono, plik TOML	RFC 3339 w API i argumenty CLI. Jeden sposób konfiguracji nie wymaga dodatkowego schematu pliku.
Przechowywanie	std::fs i własny kodek	RocksDB, SQLite	Własny silnik jest celem pracy. Format jawny i testowalny; większa odpowiedzialność za poprawność recovery.
Współbieżność	Arc, RwLock, Mutex i kanały	Spinlocki, mapy bez blokad	Osobne TS, bariera head, bufor WAL i watch postępu. Część pamięciowa działa w puli, a oczekiwanie na trwałość asynchronicznie.
Sumy i błędy	crc32fast 1, thiserror 2	Ręczny CRC, błędy tekstowe	Gotowa implementacja CRC i typowane kategorie błędów bez własnego algorytmu sum kontrolnych.
Diagnostyka	tracing 0.1, tracing-subscriber 0.3	Własny logger	Ustrukturyzowane zdarzenia i pomiar czasu operacji; bez zapisywania pełnych treści pomiarów.
Interfejs	Grafana OSS, profil 13.2.0	Własny frontend	Wykresy, selektor czasu i provisioning [7]. Zakres zgodności trzeba sprawdzić z dokładnie tym obrazem.
Wdrożenie	Docker Compose v2, Linux	Ręczna instalacja, Kubernetes	Jedna sieć, deklarowane usługi i trwałe wolumeny [11]. Orkiestrator klastra nie jest potrzebny.
Testy	cargo test, Tower 0.5; generator w Rust	Zewnętrzny framework obciążeniowy	Testy modułowe i HTTP bez osobnego stosu technologicznego; generator korzysta z reqwest 0.12 i stałego ziarna losowania.

Numery bibliotek oznaczają wybrane linie zgodności, a nie polecenie aktualizacji istniejącego manifestu zależności. Dokładne wersje implementacji utrwala `Cargo.lock`, toolchain plik `rust-toolchain.toml`, a obraz Grafany jawny tag `grafana/grafana:13.2.0`. Przy przygotowaniu

demo zapisuje się też digest obrazu. Dokument nie dodaje tych zależności ani środowiska do kodu. Docelowa walidacja odbywa się na Linux x86-64; inne systemy mogą korzystać z kontenerów linuksowych.

9.2 Konfiguracja procesu

Źródłem konfiguracji są argumenty CLI; brak parametru oznacza wartość domyślną. Compose przekazuje je jako tablicę `command`. Nie wprowadza się równoległych zmiennych środowiskowych dla tych samych ustawień. Zmiany wymagają restartu.

Parametr	Domyślnie	Znaczenie i walidacja
`--listen`	`127.0.0.1:8080`	Adres HTTP; w kontenerze jawnie `0.0.0.0:8080`.
`--data-dir`	`./data`	Katalog dostępny do zapisu, jeden proces na katalog.
`--buffer-duration`	`30m`	Czas do zlecenia flush; dodatnia pojedyncza jednostka czasu.
`--write-workers`	4	Liczba zadań zapisu wykonywanych równolegle, od 1 do 8; nie zmienia liczby logicznych WAL.
`--wal-durability`	`written`	Wybór none, written albo synced; żądanie może nadpisać ten poziom.
`--wal-flush-interval`	`10ms`	Dodatni interwał zapisu bez oczekujących klientów; nie jest terminem gwarantowanej trwałości.
`--wal-group-delay-us`	200	Nieujemne mikrosekundy zbierania grupy po pobudzeniu przez oczekujących.
`--wal-sync-interval`	`1s`	Dodatni okres fdatasync; Synced wyzwala wcześniejszą synchronizację.
`--wal-segment-bytes`	134217728	128 MiB; co najmniej 8388640 B, aby zmieścić maksymalną ramkę i nagłówek.
`--max-wal-buffer-bytes`	33554432	32 MiB na bufor przyjmujący i przejęty przez flusher, minimum 16 MiB.
`--wal-wait-timeout`	`30s`	Dodatni timeout odpowiedzi Written/Synced; nie wycofuje head.
`--max-buffer-bytes`	536870912	512 MiB; dodatni, co najmniej 65536 B. Wartość 0 wyłącza limit próbek, lecz nie pozostałe limity.
`--out-of-order-window`	`5m`	Nieujemny, nie większy od czasu buforowania. Nadal obowiązuje $t > S_s$.
`--max-future-skew`	`60s`	Dopuszczalne wyprzedzenie zegara, nieujemne.
`--lookback-delta`	`5m`	Dodatni zakres wyboru ostatniej próbki.

Parametr	Domyślnie	Znaczenie i walidacja
`--max-series`	10000	Maksymalna liczba szeregów w katalogu; liczba dodatnia.
`--max-metadata-bytes`	67108864	64 MiB na katalog i indeksy, z uwzględnieniem kopii katalogu dla checkpointu.
`--query-timeout`	`30s`	Dodatni limit pojedynczego zapytania.
`--log-level`	`info`	Jeden z: error, warn, info, debug, trace.

Domyślne, stałe limity ochronne z poprzednich sekcji tworzą profil v1: 4 MiB wejścia, 10 000 punktów paczki, 8 przyjętych żądań zapisu, dwa zapytania, 64 MiB na zapytanie i 8 MiB całej ramki WAL. Łączny budżet surowych ciał i zdekodowanych paczek oraz przygotowanych ramek WAL wynosi 64 MiB. Obejmuje także paczki aktualnie przetwarzane. Pamięć jest rezerwowana przed dekodowaniem i przy zmianie reprezentacji; przekroczenie budżetu daje 503 przed enqueue. Osobne 32 MiB WAL obejmuje kopie już przekazane do bufora i flushera. Odczyt ciała ma timeout 10 s.

Limit metadanych sprawdza się przed rejestracją szeregu oraz przed utrwaleniem kolejnej paczki, uwzględniając miejsce na indeks następnego zrzutu. Brak miejsca odrzuca nową paczkę, zamiast przyjmować dane, których zaplanowany flush przekroczyłby limit. Operator może podnieść limit po restarcie, co zmienia założenia zużycia RAM. Wyłączenie limitu bufora nie wyłącza kontroli przepełnień ani błędów alokacji.

9.3 Sieć, trwałość i granica dostępu

Usługi `tsdb`, `grafana` i `generator` są w sieci `tsdb-net`. Grafana odwołuje się do `http://tsdb:8080`, a generator zapisuje do tego samego adresu. Nie używają `localhost` do komunikacji między kontenerami. Na hosta publikowane są porty `127.0.0.1:3000:3000` i, na potrzeby testów HTTP, `127.0.0.1:8080:8080`.

Wolumen `tsdb-data` jest montowany w `/var/lib/tsdb`. Konfiguracja źródła Grafany w YAML i dashboard w JSON są montowane tylko do odczytu. Baza działa jako użytkownik bez uprawnień root z prawem zapisu do własnego wolumenu. Grafana wymaga logowania; hasło administratora przekazywane jest przy uruchomieniu przez nieśledzony plik środowiskowy.

Własne API v1 nie realizuje uwierzytelniania ani TLS: granicą dostępu jest lokalny host i zaufana sieć demonstracyjna. Udostępnienie poza nią wymaga osobnej decyzji wdrożeniowej i warstwy dostępowej. Przeglądarka nie łączy się z TSDB, więc integracja nie wymaga CORS.

Compose sprawdza `/readyz` przed uruchomieniem generatora. Po restarcie bazy generator ponawia połączenia, a Grafana odzyskuje odczyt pod tym samym adresem. Zatrzymanie kontenerów zachowuje wolumen; jego usunięcie jest osobną, świadomą operacją operatora.

9.4 Diagnostyka

Logi zawierają identyfikator żądania, endpoint, wynik, czas obsługi, LSN, wybrany tryb trwałości i liczniki punktów. Dla zrzutu zapisuje się generację head, checkpoint i kursor replay, liczbę punktów, bajty i czas. Recovery raportuje czas, liczbę odtworzonych ramek oraz ewentualne obcięcie niepełnego ogona. Nie zapisuje się pełnych ciał żądań ani wartości etykiet w każdym logu.

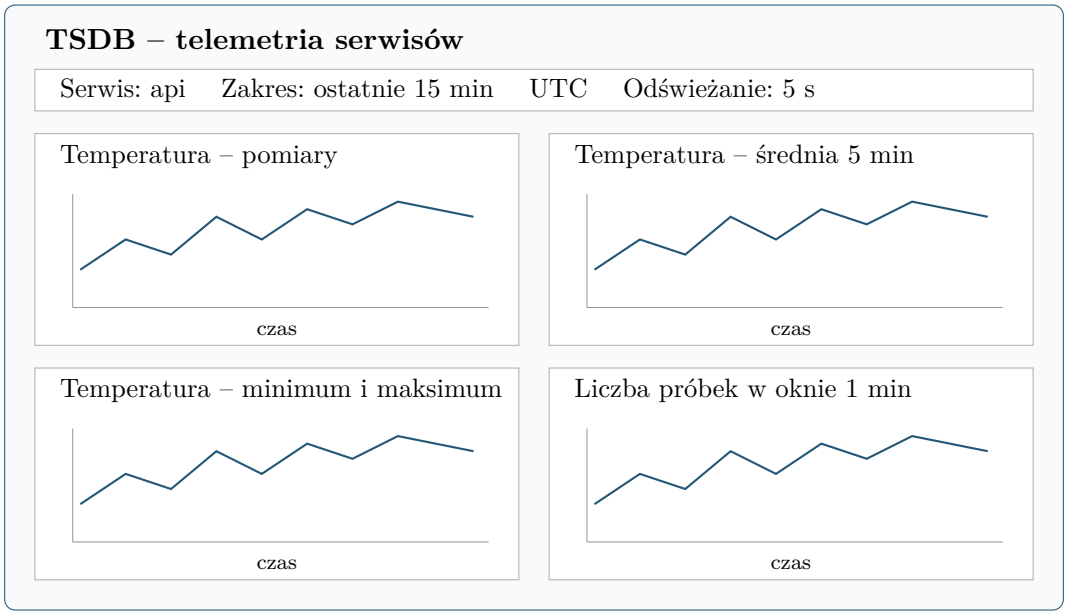
Do eksperymentów zbiera się RSS z systemu operacyjnego, rozmiar bufora, pojemność obu buforów

WAL, różnice enqueued/written/synced, czas write i fdatasync, długość kolejki i czas flush. Wewnętrzne liczniki mogą być raportowane okresowo w logach; nie wymagają uruchamiania kolejnego systemu monitorowania.

10 Wizja interfejsu użytkownika i demonstracja

10.1 Dashboard Grafany

Dashboard „TSDB – telemetria serwisów” zawiera wybór jednego serwisu, selektor czasu, automatyczne odświeżanie co 5 s i cztery panele. Początkowy zakres to ostatnie 15 min, strefa wyświetlania UTC. Źródła generatora mają etykiety `service` i `instance`; różne instancje są osobnymi liniami na wykresie.



Rysunek 9: Makieta układu dashboardu. Linie są ilustracją, nie wynikami eksperymentu.

Panel	Wyrażenie projektowanego dashboardu
Pomiary	`temperature_celsius{service="$service"}`
Średnia 5 min	`avg_over_time(temperature_celsius{service="$service"}[5m])`
Minimum i maksimum	Dwa zapytania: `min_over_time(temperature_celsius{service="$service"}[5m])` oraz `max_over_time(temperature_celsius{service="$service"}[5m])`.
Liczba próbek	`count_over_time(temperature_celsius{service="$service"}[1m])`

Zmienna `service` jest typu *Custom*, z wartościami `api` i `worker`. Wyłączone są opcje wielokrotnego wyboru i „All”, aby Grafana nie generowała regexów. Zapytania są zapisane w trybie *Code*. Minimalny krok panelu to 1 s; Grafana może go zwiększać dla dłuższego zakresu. Nie używa się makra generującego funkcje spoza podzbioru.

Brak danych jest przedstawiany jako przerwa lub „No data”, bez łączenia luk wypełnieniem zerami. Błąd API pozostaje widocznym błędem panelu, a nie pustym poprawnym wynikiem. Panel liczby próbek nie jest licznikiem wszystkich szeregów ani agregacją między instancjami.

10.2 Profil standardowego źródła Prometheus

Provisioning ustawia stały UID, adres bazy i metodę POST. Poniższy fragment jest projektowaną konfiguracją do użycia podczas implementacji demo; znaczenie ustawień źródła wynika z dokumentacji Grafany [6].

```
apiVersion: 1
datasources:
  - name: TSDB
    uid: tsdb
    type: prometheus
    access: proxy
    url: http://tsdb:8080
    isDefault: true
    editable: false
    jsonData:
      httpMethod: POST
      timeInterval: 1s
      manageAlerts: false
      disableRecordingRules: true
      prometheusType: Prometheus
      prometheusVersion: 2.0.0
```

`prometheusVersion:2.0.0` wybiera konserwatywny profil funkcji klienta, nie deklaruje wersji silnika TSDB. Prawdziwą wersję zwraca `buildinfo`. Wyłącza się incremental querying, exemplary i funkcje edytora wymagające nieobsługiwanych endpointów. Provisioning dashboardu wskazuje katalog plików JSON i UID `tsdb`. Brakującą funkcję Grafany weryfikuje się na przypiętym obrazie, bez deklarowania zgodności ze wszystkimi wersjami.

10.3 Scenariusz demonstracji

1. Uruchomić Compose z pustym lub istniejącym wolumenem. Poczekać na gotowość bazy i uruchomienie generatorów.
2. Generatory `api` i `worker` wysyłają co sekundę temperaturę dwóch instancji. Jeden profil celowo wprowadza opóźnione punkty.
3. Otworzyć dashboard, zmienić serwis i zakres czasu; porównać pomiary z agregacjami i liczbą próbek.
4. W profilu demonstracyjnym skrócić buforowanie do 1 min, a okno opóźnienia do 30 s. Pokazać potwierdzane zapisy i odczyty podczas flush, niezmienność wcześniejszych danych po podmianie reprezentacji oraz błąd punktu skierowanego do zamrożonego zakresu.
5. Zrestartować bazę z zachowaniem wolumenu. Pokazać wcześniejsze dane i dalszy napływ pomiarów.

Profil pokazowy z krótkim buforem nie jest profilem testu zasobów z lab1. Scenariusz awarii z SIGKILL jest wykonywany w testach kontrolowanych; sam restart demonstracyjny nie dowodzi gwarancji NFR-04.

11 Weryfikacja architektury i kryteria odbioru

11.1 Powiązanie z wymaganiami

Wymaganie	Mechanizm	Weryfikacja
FR-01	HTTP, walidacja, atomowa ramka WAL.	T-01: zapis pojedynczy i paczka; błędna paczka nie zmienia danych.
FR-02	Kanoniczny klucz i katalog szeregów.	T-02: kolejność etykiet, inne wartości, puste etykiety i kolizje hash.
FR-03	Written/Synced, bufor WAL i checkpoint.	T-03, T-10, T-16: replay potwierdzonych Written/Synced; osobna ocena None.
FR-04	Siatka czasu i jeden ReadView.	T-04: lookback, granice, krok niedzielący zakresu i puste wyniki.
FR-05	Parser ograniczonej gramatyki.	T-05: wszystkie konstrukcje, pełne zużycie wejścia, odrzucane rozszerzenia.
FR-06	Agregacje osobno dla szeregu.	T-06: pięć funkcji, granice okna, kilka szeregów, tolerancja błędu f64.
FR-07	API Prometheus i provisioning.	T-07: dashboard i test połączenia na przypiętym obrazie Grafany.
FR-08	Rotacja bloków, budżet pamięci i flush w tle.	T-08, T-14–15: timer, nowe zapisy podczas flush i pamięć utrzymywana przez odczyt.
FR-09	W, H_s, S_s.	T-09: granice równości, flush, zmiana konfiguracji i restart.
NFR-01–03	Paczki, indeksy, limity i kontrola alokacji.	P-01–03: testy HTTP, przepustowości i RSS z opisanym profilem.
NFR-04	Recovery i kolejność publikacji plików.	T-10 i P-04: wymuszone awarie i czas gotowości.
NFR-05	Typowane błędy i walidacja.	T-11: macierz błędnych żądań bez zakończenia procesu.
DA-02–04, DA-08	TSMap, blokady TS, bariera rotacji i jeden WAL.	T-12–15 i P-05: równoległość, brak zakleszczeń i poprawność widoków.
DA-07, DA-09	Flusher, syncer, granice prefiksów i tryby per żądanie.	T-16–20 i P-06: przyczynowość definicji, rotacja, checkpoint i błędy I/O.

11.2 Scenariusze poprawności

T-01: Zapis pojedynczy, wieloszeregowa paczka i 10 000 punktów. Odczyt po odpowiedzi widzi całość. Błąd jednego punktu oznacza brak zmian w próbkach i publicznym katalogu; niewidoczne luki ID są dopuszczalne.

T-02: Zamieniona kolejność etykiet daje ten sam szereg; zmieniona wartość daje inny. Puste etykiety są usuwane. Powtórzone klucze JSON i zastrzeżone nazwy są odrzucane.

T-03: Restart z danymi tylko w WAL, z historią w segmentach i pustym aktywnym WAL oraz ze stanem mieszanym. Sprawdzenie próbek, etykiet, kolejnych identyfikatorów i granic zamknięcia.

T-04: Zapytania chwilowe i zakresowe: $start = end$, zerowy krok, odwrócony zakres, końcowy punkt poza siatką, próbka dokładnie na granicy lookback, kilka szeregów i brak danych.

T-05: Parser: każda funkcja, escape'y napisów, białe znaki, skalar `1+1`, błędna jednostka, trailing input, regex i nieobsługiwana funkcja.

T-06: Dane z ręcznie policzonymi wynikami, puste okno, granice $(T - R, T]$, skrajne skończone wartości i wynik nieskończony. Porównanie RAM, segmentu i mieszanego widoku. Wyniki

f64 sprawdzane z tolerancją $10^{-9}\max(1, |wynik_{\mathrm{ref}}|)$.

T-07: Uruchomienie ustalonego obrazu Grafany, provisioning i rzeczywiste żądania wszystkich paneli. Zmiana serwisu i czasu, przerwa danych, restart bazy oraz próba połączenia źródła.

T-08: Flush czasowy i pamięciowy, także przy aktywnych odczytach. Odczyt widzi próbkę raz; zrzut nie robi pełnej kopii RAM. Za mały limit na pojedynczą paczkę nie wywołuje pętli flush.

T-09: Czasy na granicy $H - W$ i S, po obu stronach granic, duplikaty zgodne i konfliktowe. Ponowna walidacja po wymuszonym flush. Replay po zmianie konfiguracji nie odrzuca potwierdzonych punktów.

T-10: Co najmniej 20 awarii procesu w każdym z trybów Written i Synced, z osobnym dziennikiem potwierdzeń klienta. Awarie w środku ramki, po write i sync, przy rename segmentu i manifestu, przed i po rotacji WAL oraz usunięciu plików sprzed kotwicy. Każda potwierdzona paczka Written/Synced musi wrócić. None i paczki bez odpowiedzi mogą być obecne w całości albo nieobecne.

T-11: Błędne typy, NaN w wejściu, zbyt duże ciało, nieznane parametry, pełna kolejka, zajęty semafor zapytań oraz timeout oczekiwania na WAL. Sprawdzenie kodu, opisu błędu i możliwości dalszej diagnostyki procesu.

T-12: Co najmniej dwa zadania zapisu rozłącznych TS. Wstrzymać syncer i pozostawić klienta Synced w oczekiwaniu po publikacji. Inny zapis None, także do tego samego TS, i odczyt mają móc się zakończyć. Dla tego samego TS konkurujące duplikaty i konflikty muszą dać wynik zgodny z ustaloną kolejnością, bez utraty próbek.

T-13: Dwie paczki blokujące te same TS podane w odwrotnej kolejności wejścia nie zakleszczają się. Odczyt kilku TS widzi całą paczkę albo stan sprzed niej. Równoległa rejestracja tego samego klucza tworzy jeden TS; zmiana wersji katalogu podczas tworzenia widoku wymusza ponowienie wyboru.

T-14: Wstrzymać zadanie flush po rotacji, przed trwałym segmentem. Potwierdzić nowe zapisy do aktywnych bloków i odczytać dane obu generacji. Po publikacji segmentu wynik pozostaje taki sam, a WAL nowych zapisów nie zostaje usunięty. Powtórzyć restart przed i po publikacji manifestu, również z nowym szeregiem zarejestrowanym podczas flush.

T-15: Powolny flush, aktywny bufor bliski limitu i ReadView podtrzymujący stary blok po publikacji segmentu. Pamięć pozostaje rozliczona do ostatniego uchwytu. Przeciążenie daje 503 przed enqueue, nie tworzy kolejnych zamrożonych generacji i nie blokuje operacji zwalniających miejsce.

T-16: Kontrolowane zatrzymanie flushera i checkpointu: None odpowiada, a Written i Synced czekają mimo widoczności head. Po write kończy się Written, po fdatasync Synced. Próba awarii przed write dopuszcza utratę None. Ponowienie identycznej próbki None z Synced musi naprawdę czekać na sync.

T-17: Wstrzymać rejestrację szeregu przed enqueue definicji. Konkurent nie może wcześniej zalogować jego próbek. Po zwolnieniu rejestracji potwierdzić drugi zapis jako Synced, zrestartować proces i odczytać definicję oraz oba punkty. Rekord z nieznanym ID zatrzymuje replay, nie zwiększa licznika pominięć.

T-18: Zatrzymać syncer po zapamiętaniu celu, dopisać późniejsze rekordy i zakończyć pierwszą synchronizację. synced_end może potwierdzić tylko zapamiętany prefiks. Powtórzyć przy rotacji pliku, krótkim write, anulowaniu i rejestracji waitera po wcześniejszym postępie watch.

T-19: Rekord jest w WAL, ale jego head append jest wstrzymany: bariera checkpointu ma czekać na publikację, nie na optional wait. Kursor w środku pliku i dokładnie na końcu

pliku daje poprawny replay. Sprawdzić utrzymanie kotwicy, aktywnego pliku i rekordów po C. Wiele rotacji WAL bez flush oraz wiele flush bez rotacji WAL nie zmienia wyniku odczytu.

T-20: Wypełnić budżet aktywnego i przejętego bufora WAL. Odrzucenie następuje przed enqueue, bez częściowej paczki i bez nieograniczonej kolejki. Zasymulowany błąd write/fdatasync kończy proces bez sukcesu nieosiągniętych Written/Synced; wcześniej potwierdzone dane pozostają odtwarzalne przy sprawnym nośniku testowym.

Testy kodeków obejmują znane sekwencje bajtów, poprawne długości i CRC, każdy możliwy punkt ucięcia małej ramki, nadmierne liczniki, offset poza plikiem, uszkodzony nagłówek i pełną ramkę z błędnym CRC. Ostatni przypadek ma zatrzymać start, a nie być uznany za niepełny ogon. Scenariusze współbieżne wykorzystują bariery testowe do wymuszenia konkretnych przeplotów operacji. Sprawdzają również ciągłość LSN między plikami, pusty nowy WAL, niepełny ogon wyłącznie ostatniego pliku oraz odtwarzanie po kursorze bajtowym przy niezależnych generacjach head.

11.3 Profil pomiarów wydajności i zasobów

Wyniki są raportowane dla konkretnego sprzętu: model CPU, liczba rdzeni, RAM, typ dysku i system plików, wersja systemu, toolchain, commit, ustawienia bazy oraz współbieżność klienta. Profil referencyjny: Linux x86-64, co najmniej 4 rdzenie, 8 GiB RAM, lokalny SSD, kompilacja release, bez innego obciążenia dysku. Podanie tych parametrów umożliwia interpretację wyników; nie zastępuje pomiaru.

Test	Przebieg	Cel z lab1
P-01	Po 30 s rozgrzewki co najmniej 5 min pomiaru. Osobne scenariusze zapisu jednego punktu, paczki 1000 punktów, odczytu 1000 wyników i agregacji. Profil bazowy Written; czas HTTP obejmuje kolejkę, publikację head i oczekiwanie na write.	p95 zapisu punktu ≤ 100 ms; paczki ≤ 500 ms; odczytu ≤ 200 ms; agregacji z RAM ≤ 500 ms.
P-02	1000 szeregów o stałych etykietach, paczki po 1000 punktów, 4 współbieżnych klientów, Written. Co najmniej godzina, by objąć flush; raportować także najgorsze okna 1 s i 10 s.	Średnio $\geq 10\,000$ nowych potwierdzonych punktów/s, z uwzględnieniem rotacji, działania flush i błędów.
P-03	10 000 punktów/s, 1000 szeregów, buforowanie 30 min, domyślne limity, minimum 65 min. Bazowy scenariusz bez zapytań, dodatkowo wariant z dwoma zapytaniami.	RSS procesu ≤ 1 GiB, w tym szczyt przy flush i realokacji. Raportować rzeczywisty czas przebywania punktów w RAM.
P-04	Milion potwierdzonych punktów; osobno stan z przewagą WAL i stan po flush. Pomiar od startu procesu do pierwszego 200 z readyz; osobno cache zimny i ciepły.	Gotowość w ≤ 10 s i brak utraty danych w T-10.
P-05	Warianty 1, 2, 4 i 8 zadań zapisu i klientów. Osobno rozłączne szeregi oraz wszystkie paczki kierowane do jednego TS. Pomiar obejmuje co najmniej jeden flush.	Ocena rzeczywistej równoległości, kosztu blokad i wspólnego WAL; bez założenia liniowego przyspieszenia.

Test	Przebieg	Cel z lab1
P-06	Osobne przebiegi None, Written i Synced dla tych samych paczek. Zmiana opóźnienia grupy i obserwacja opóźnień flushera/syncera.	Raportować p95, przepustowość i bajty oczekujące oddzielnie; None nie zalicza kryterium trwałej przepustowości.

Dla P-01 zapytanie selektora obejmuje jeden szereg i 1000 chwil co 1 s; generator dostarcza próbki co 1 s. Agregacja obejmuje jeden szereg, okno 5 min i tę samą siatkę; potrzebny zakres jest w RAM. Dla P-02 i P-03 generator dostarcza po 10 punktów/s na każdy z 1000 szeregów. Etykiety mają najwyżej cztery krótkie pary. Bazowy profil ma 0% out-of-order; dodatkowe przebiegi mają 1% i 10% opóźnionych próbek z opóźnieniem do 5 min. Odrzucenia po zamknięciu zakresu są raportowane, a nie zaliczane do przepustowości.

Przy 10 000 punktów/s i 30 min powstaje:

$$10\,000 \cdot 1800 = 18\,000\,000 \text{ punktów}, \qquad 18\,000\,000 \cdot 16 = 288\,000\,000 \text{ B} \approx 274{,}7 \text{ MiB}.$$

Do tego dochodzi zapas pojemności, jednoczesna obecność bloków aktywnych i zamrożonych, uchwyty odczytów, metadane, kolejka, przygotowane WAL, obsługa HTTP i budżety zapytań. Domyślne 512 MiB bufora pozostawia margines do celu RSS, ale **nie dowodzi** jego spełnienia. Nie wolno utożsamiać limitu bufora z limitem całego procesu. Wydajność wspólnego WAL, czas oczekiwania na blokady TS i długość bariery rotacji wymagają pomiaru. Osobno raportuje się czas dyskowego flush, który nie jest już globalną pauzą zapisu. Jeśli cel nie zostanie osiągnięty, raport pokazuje wynik i wąskie gardło.

11.4 Ryzyka i warunki akceptacji projektu

Ryzyko	Sposób ograniczenia	Obserwowany wskaźnik
Flush zużywa zapas pamięci	Rotacja przy 80% budżetu, jeden flush w tle, rozliczanie bloków podtrzymywanych przez odczyty i backpressure.	Czas flush i bariery, pamięć obu generacji, 503 i p95 zapisu.
Konkurencja o blokady i WAL	Krótkie blokady TS, enqueue bez I/O i oczekiwanie na Written/Synced dopiero po publikacji.	T-12–13, P-05–06, czas części pamięciowej i opcjonalnego oczekiwania.
Koszt out-of-order	Wektor uporządkowany, jawny limit W i zamknięta granica; test udziału opóźnionych danych.	Czas wstawienia i liczba przesuniętych próbek.
Wzrost indeksów i plików	Limit katalogu i metadanych; jawny brak retencji i kompaktowania w v1.	Liczba szeregów, segmentów, bajty metadanych i dysku.
Niepełna zgodność Grafany	Ustalony obraz, tryb Code, ograniczone zmienne i wczesny test rzeczywistych żądań.	T-07 oraz lista wywołanych endpointów i parametrów.
Błąd checkpointu	Manifest jako jedyne źródło prawdy, sync przed rename, WAL usuwany na końcu.	T-10 i testy uszkodzeń plików.

Akceptacja projektu architektury wymaga spójności kontraktów i formatów, pokrycia wszystkich obszarów wytycznych lab2 oraz prześledzenia scenariuszy awarii. Akceptacja późniejszej implementacji wymaga wykonania testów i pomiarów; niniejszy dokument nie oznacza ich zaliczenia.

A Słownik

Pojęcie	Znaczenie
WAL	Rekord jest włączany do uporządkowanego bufora przed zmianą head; moment trwałego zapisu zależy od wybranego trybu.
LSN	Koniec ramki w logicznym strumieniu bajtów WAL, bez nagłówków plików.
Head	Katalog i próbki w pamięci, obejmujące aktywne i zamrożone bloki TS.
Flusher	Przenosi bufory WAL do systemu operacyjnego i publikuje written_end.
Syncer	Synchronizuje zapisany prefiks WAL i publikuje synced_end.
TSMap	Jeden katalog kluczy i uchwytów TS, z indeksem etykiet.
TS	Stan jednego szeregu, chroniony własną blokadą, z blokiem aktywnym i referencjami do danych zamrożonych i dyskowych.
Generacja	Próbki head wspólnie zamrażane do jednego zrzutu. Numeracja jest niezależna od plików WAL.
Segment	Niezmienny plik próbek i indeksów utworzony przez flush.
Manifest	Trwały spis opublikowanych segmentów, katalog szeregów i checkpoint.
Checkpoint	Największy LSN, którego wszystkie dane znajdują się w segmentach wskazanych przez manifest.
Zamrożenie head	Krótka bariera kończąca części pamięciowe zapisów i wyznaczająca granicę checkpointu.
Rotacja WAL	Zamknięcie pełnego pliku i otwarcie następnego, niezależne od zamrażania head.
Kotwica replay	Zachowany plik WAL i offset, od których recovery wznawia odtwarzanie po checkpoint.
Flush	Zapis zamrożonej generacji do segmentu i publikacja manifestu w tle.
Out-of-order	Przyjęcie próbki starszej niż najnowsza próbka danego szeregu.
Granica zamknięcia	Czas S_s; nowe próbki muszą mieć czas większy od tej granicy.
Lookback	Maksymalny wiek próbki wybieranej przez selektor chwilowy.
Provisioning	Deklaratywne dostarczenie źródła danych i dashboardu Grafany.
RSS	Pamięć rezydentna procesu; obejmuje więcej niż surowe próbki bufora.

B Budowanie dokumentu

Źródło jest samodzielnym plikiem LaTeX. Diagramy i makieta są zapisane w TikZ, bez zależności od zewnętrznych obrazów lub usług. Wymagane są pdfLaTeX, latexmk oraz pakiety użyte w preambule, w szczególności obsługa języka polskiego i TikZ. Z katalogu głównego repozytorium:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  -outdir=target/lab2-doc \
  pz/lab2/lab2-Blonowski-Zaborski.tex
```

Pliki pomocnicze trafiają do ignorowanego katalogu `target/lab2-doc`. PDF do oddania jest kopią wyniku kompilacji umieszczoną obok źródła. Po zmianach należy sprawdzić ostrzeżenia składu, odwołania, spis treści oraz czytelność tabel i diagramów.

Literatura

[1] H. Błonowski, J. Zaborski, *Etap 1: Specyfikacja wymagań i zakres pracy*, wersja 1.1, 21.10.2026, dokument projektu w `pz/lab1`.

[2] T. Pelkonen, S. Franklin, P. Cavallaro, Q. Huang, J. Meza, J. Teller, K. Veeraraghavan, *Gorilla: A Fast, Scalable, In-Memory Time Series Database*, Proceedings of the VLDB Endowment, 8(12), s. 1816–1827, 2015, sekcje 4.2–4.3, `https://www.vldb.org/pvldb/vol8/p1816-teller.pdf`.

[3] Prometheus, *HTTP API*, `https://prometheus.io/docs/prometheus/latest/querying/api/`.

[4] Prometheus, *Querying basics*, `https://prometheus.io/docs/prometheus/latest/querying/basics/`.

[5] Prometheus, *Query functions*, `https://prometheus.io/docs/prometheus/latest/querying/functions/`.

[6] Grafana Labs, *Configure the Prometheus data source*, `https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/`.

[7] Grafana Labs, *Provision Grafana*, `https://grafana.com/docs/grafana/latest/administration/provisioning/`.

[8] Projekt Axum, *Dokumentacja biblioteki*, `https://docs.rs/axum/latest/axum/`.

[9] Projekt Rust, *std::fs::File*, `https://doc.rust-lang.org/std/fs/struct.File.html`.

[10] Projekt crc32fast, *Dokumentacja biblioteki*, `https://docs.rs/crc32fast/latest/crc32fast/`.

[11] Docker, *How Compose works*, `https://docs.docker.com/compose/intro/compose-application-model/`.

[12] Linux man-pages, *write(2)*, `https://man7.org/linux/man-pages/man2/write.2.html`.

[13] Linux man-pages, *fsync(2), fdatasync(2)*, `https://man7.org/linux/man-pages/man2/fsync.2.html`.

[14] Linux man-pages, *sync_file_range(2)*, `https://man7.org/linux/man-pages/man2/sync_file_range.2.html`.

[15] Projekt Tokio, *tokio::sync::watch*, `https://docs.rs/tokio/latest/tokio/sync/watch/index.html`.

Dokumentację zewnętrzną sprawdzono 24.10.2026. Zachowanie własnego silnika określa niniejsza specyfikacja; odnośniki *latest* służą jako kontekst, a zgodność integracji sprawdza się z przypiętymi wersjami używanymi w demonstracji.